



**Universidad
Europea**

UNIVERSIDAD EUROPEA DE MADRID

**ESCUELA DE ARQUITECTURA, INGENIERÍA Y
DISEÑO**

GRADO EN FÍSICA

TRABAJO FIN DE GRADO

**APLICACIÓN DE TENSOR
NETWORKS COMO MODELOS DE
DEEP LEARNING PARA
CLASIFICACIÓN DE IMÁGENES**

MANUEL ARENAS SÁNCHEZ

Dirigido por

ALEJANDRO MATA ALI

MARÍA FUENCISLA GILSANZ MUÑOZ

CURSO 2025-2026

TÍTULO: APLICACIÓN DE TENSOR NETWORKS COMO MODELOS DE DEEP LEARNING PARA CLASIFICACIÓN DE IMÁGENES

AUTOR: MANUEL ARENAS SÁNCHEZ

TITULACIÓN: GRADO EN FÍSICA

DIRECTOR/ES DEL PROYECTO: ALEJANDRO MATA ALI
MARÍA FUENCISLA GILSANZ MUÑOZ

FECHA: MARZO DE 2026

RESUMEN

Este Trabajo de Fin de Grado estudia la aplicación de las *Tensor Networks* en modelos de *deep learning* para clasificación de imágenes. Las redes tensoriales proceden originalmente de la física cuántica de muchos cuerpos y del álgebra multilineal, pero en los últimos años también se han explorado como arquitecturas entrenables en problemas de *machine learning*. En este contexto, el objetivo del trabajo es analizar si las redes tensoriales pueden integrarse de forma práctica en un problema de clasificación de imágenes, y qué limitaciones aparecen durante su desarrollo.

Para ello se ha diseñado e implementado un *pipeline* completo de clasificación de imágenes en PyTorch. El trabajo incluye la preparación del conjunto de datos, la generación de recortes a partir de anotaciones de detección, el preprocesamiento de imágenes, la definición configurable de modelos, el entrenamiento, la evaluación y el registro de experimentos. El conjunto de datos principal utilizado es ChemEq25. Está formado por imágenes reales de material de laboratorio. Los entrenamientos se realizaron en los servidores de la Universidad Europea de Madrid (Lorca).

La arquitectura desarrollada combina una etapa inicial de extracción de características visuales con una red tensorial jerárquica de tipo *Tree Tensor Network*. La primera parte transforma cada imagen en una representación adecuada para el modelo, conservando información espacial relevante. Posteriormente, la red tensorial procesa esa representación mediante bloques estructurados y produce una predicción de clase. A lo largo del proyecto se estudiaron distintas decisiones de diseño, como la representación de entrada, la dimensión interna del modelo, la regularización, el aumento de datos y la estabilidad del entrenamiento.

Los resultados muestran que la arquitectura tensorial propuesta puede aprender la tarea de clasificación sobre imágenes reales cuando se combina con una representación de entrada adecuada y un proceso de entrenamiento bien controlado. En los experimentos realizados sobre ChemEq25, el mejor modelo obtuvo un 96,05 % de precisión global, un 95,94 % de F1 macro y un 99,43 % de precisión top-5. Estos resultados deben interpretarse de forma crítica. El rendimiento no depende únicamente de la topología tensorial, sino también del preprocesamiento, la extracción inicial de características, la regularización y la trazabilidad experimental. En conjunto, el trabajo aporta una implementación funcional y reproducible de un modelo basado en redes tensoriales para clasificación de imágenes, así como un análisis de sus posibilidades y limitaciones frente a enfoques convencionales de visión artificial.

Palabras clave: redes tensoriales, aprendizaje profundo, clasificación de imágenes, Tree Tensor Network, PyTorch, ChemEq25.

ABSTRACT

This Bachelor's Thesis studies the application of *Tensor Networks* to deep learning models for image classification. Tensor networks were originally developed in the context of quantum many-body physics and multilinear algebra, but in recent years they have also been explored as trainable architectures in machine learning problems. In this context, the aim of this work is to analyse if tensor networks can be practically integrated into image classification task and what limitations arise during their development.

To this end, a complete image classification pipeline has been designed and implemented in PyTorch. The project includes dataset preparation, crop generation from detection annotations, image preprocessing, configurable model definition, training, evaluation and experiment logging. The main dataset used is ChemEq25, which contains real images of laboratory equipment. The training experiments were carried out on the computing servers of Universidad Europea de Madrid (Lorca).

The proposed architecture combines an initial visual feature extraction stage with a hierarchical tensor network of the *Tree Tensor Network* type. The first part transforms each image into a representation suitable for the model while preserving relevant spatial information. The tensor network then processes this representation through structured blocks and produces a class prediction. Throughout the project, several design choices were analysed, including the input representation, the internal model dimension, regularisation, data augmentation and training stability.

The results show that the proposed tensor-based architecture can learn the classification task on real images when it is combined with an appropriate input representation and a well-controlled training procedure. In the experiments carried out on ChemEq25, the best model achieved 96.05 % overall accuracy, 95.94 % macro F1 and 99.43 % top-5 accuracy. These results should be interpreted critically. Performance does not depend only on the tensor topology, but also on preprocessing, the initial feature extraction stage, regularisation and experimental traceability. Overall, this work provides a functional and reproducible implementation of a tensor-network-based model for image classification, together with an analysis of its possibilities and limitations compared with conventional computer vision approaches.

Keywords: tensor networks, deep learning, image classification, Tree Tensor Network, PyTorch, ChemEq25.

AGRADECIMIENTOS

En primer lugar, me gustaría dar las gracias a Alejandro Mata Ali por toda la ayuda que me ha prestado. Ha seguido mi progreso de cerca en todo momento, me ha facilitado una gran cantidad de material de estudio y me ha ofrecido nuevas perspectivas siempre que me he atascado en algún problema.

Quiero agradecer también a María Fuencisla Gilsanz Muñoz, mi tutora en la UEM, que igualmente ha acompañado mi trabajo en todo momento y me ha aportado numerosas ideas y mucho material durante la realización de este TFG.

También quiero agradecer a todas las personas con las que he trabajado en el Instituto Tecnológico de Castilla y León (ITCL), y en especial al departamento de computación cuántica, formado por Sergio Muñiz Subiñas, Miguel Franco Hernando, Jorge Martínez Martín y Alejandro Mata Ali. Ellos han sido una pieza fundamental, pues gracias a su acompañamiento he aprendido y disfrutado enormemente de mis prácticas en el ITCL, además de recibir muchos consejos sobre el desarrollo de este proyecto. Sin ellos, nada de esto habría sido lo mismo.

Quisiera dedicar una mención muy especial al profesor Iñaki Ortiz Delandaluce, que siempre ha creído en mí y cuyo apoyo incondicional ha supuesto para mí una motivación enorme. Sin él, probablemente no habría entrado en el ITCL y, muy posiblemente, este TFG no existiría.

Por último, quiero agradecer a las personas más importantes para mí: mi familia; que me han apoyado siempre a lo largo de toda la carrera de física, en la que hemos atravesado momentos muy difíciles. Siempre creyeron en mí y me dieron fuerzas hasta el final.

Cita - frase célebre / Dedicatoria

“It is not our part to master all the tides of the world, but to do what is in us for the succour of those years wherein we are set, uprooting the evil in the fields that we know, so that those who live after may have clean earth to till. What weather they shall have is not ours to rule.”

J.R.R. Tolkien

TABLA RESUMEN

	DATOS
Nombre y apellidos:	Manuel Arenas Sánchez
Título del proyecto:	Aplicación de Tensor Networks en modelos de IA para clasificación de imágenes.
Directores del proyecto:	Alejandro Mata Ali y María Fuencisla Gilsanz
El proyecto se ha realizado en colaboración de una empresa o a petición de una empresa:	SI
El proyecto ha implementado un producto: (esta entrada se puede marcar junto a la siguiente)	NO
El proyecto ha consistido en el desarrollo de una investigación o innovación: (esta entrada se puede marcar junto a la anterior)	SI
Objetivo general del proyecto:	Estudiar la implementación de las <i>tensor networks</i> en <i>deep learning</i>

ÍNDICE

RESUMEN	4
ABSTRACT	5
AGRADECIMIENTOS	6
TABLA RESUMEN	8
1 INTRODUCCIÓN	15
1.1 Estado del Arte	15
1.2 Planteamiento del Problema	17
1.3 Planificación del TFG	19
1.4 Recursos requeridos	21
2 OBJETIVOS	23
2.1 Objetivos generales	23
2.2 Objetivos específicos	23
2.3 Objetivos que quedan fuera del proyecto	24
2.4 Beneficios del proyecto	24
3 MARCO TEÓRICO	26
3.1 Deep learning y clasificación de imágenes	26
3.2 Tensores	28
3.3 Notación diagramática de Penrose y definición de Tensor Network	30
3.4 Tensor Networks y reducción de dimensionalidad	33
3.5 Descomposiciones tensoriales fundamentales	34
3.6 Topologías principales de Tensor Networks	36
3.6.1 Matrix Product State / Tensor Train	36
3.6.2 Tree Tensor Network	38

3.6.3	MERA	39
3.6.4	PEPS	40
3.7	Tensor Networks en aprendizaje automático	41
4	METODOLOGÍA	43
4.1	Visión general del pipeline	43
4.2	Preparación del dataset ChemEq25	44
4.3	Transformaciones y aumento de datos	45
4.4	Representación de la imagen como secuencia	46
4.5	Arquitectura tensorial propuesta	49
4.6	Entrenamiento y regularización	51
4.7	Métricas, diagnósticos y trazabilidad	53
4.8	Comparación con ResNet50	54
4.9	Reproducibilidad y ejecución en servidor	55
5	RESULTADOS	57
5.1	Mejor modelo tensorial obtenido	57
5.2	Comportamiento del modelo tensorial	59
5.3	Comparación con ResNet50	61
6	CONCLUSIONES	66
6.1	Conclusiones del trabajo	66
6.2	Conclusiones personales	67
	GLOSARIO	69
	Acrónimos y siglas	69
	Conceptos teóricos	71
	Términos de implementación y código	73
	Componentes de la arquitectura	73
	Hiperparámetros y configuración	75
	Entrenamiento, regularización y evaluación	75
	REFERENCIAS	79

ÍNDICE DE FIGURAS

1.1	Planificación por semanas del TFG, cada círculo representa una semana teniendo febrero 3 semanas, marzo 4 semanas, abril cuatro semanas y por último mayo tendrá otras cuatro semanas. De esta manera se cumplen entre 500 y 600 horas dedicadas al TFG.	20
3.1	Esquema de una red neuronal simple. [1]	26
3.2	Esquema de topologías básicas de redes neuronales, desde el perceptrón hasta arquitecturas con capas ocultas y conexiones recurrentes [2].	28
3.3	Representación de una TN inventada. En esta representación se pueden identificar muchas de las operaciones descritas en esta sección. Elaboración propia usando la librería Tensor-Network-Editor, creada por Alejandro Mata Ali. [3]	31
3.4	Ejemplo de representación diagramática para un tensor. Fuente: [4]	32
3.5	Diagramas TN de algunas descomposiciones tensoriales populares. (a) El formato CP descompone un tensor X en una suma de varios tensores de rango 1 $a_{:,r}^{(1)} \circ a_{:,r}^{(2)} \circ \dots \circ a_{:,r}^{(N)}$. (b) La descomposición de Tucker descompone un tensor X en un tensor núcleo G multiplicado por una matriz $A^{(n)}$ a lo largo del modo n -ésimo. (d) La descomposición TT descompone un tensor X en una multiplicación lineal de un conjunto de tensores núcleo de tercer orden $G^{(2)} \dots G^{(N-1)}$ y dos matrices $G^{(1)}$ y $G^{(N)}$. (g) La descomposición en malla tensorial, también conocida como PEPS, representa un tensor de alta dimensión como una malla bidimensional de tensores interconectados de menor rango, donde cada nodo se conecta con sus vecinos para capturar eficientemente correlaciones espaciales en sistemas con interacciones locales. Los paneles (c), (e) y (f) corresponden, respectivamente, a las descomposiciones en términos de bloque, Tensor Ring (TR) y jerárquica de Tucker (HT), no empleadas en este trabajo. [5]	36
4.1	Esquema general del pipeline	44
4.2	Diagrama de la <i>Tree Tensor Network</i> propuesta	49
4.3	Captura de una GPU NVIDIA RTX 4000 Ada Generation del servidor Lorca de la UEM ejecutando dos procesos de entrenamiento en paralelo.	56

5.1	Evolución del entrenamiento del mejor modelo tensorial. En (a) se representa la función de pérdida y en (b) la precisión, comparando el comportamiento sobre entrenamiento y evaluación.	60
5.2	Matriz de confusión del mejor modelo tensorial sobre el conjunto de evaluación. La concentración de valores en la diagonal principal indica que la mayoría de los recortes se clasifican correctamente.	61
5.3	Comparación entre el modelo tensorial propuesto y ResNet50. En (a) se muestra la precisión top-1 y top-5 alcanzada por cada modelo y en (b) la relación entre la precisión top-1 y el número aproximado de parámetros entrenables.	63
5.4	Evolución del entrenamiento de ResNet50 sobre ChemEq25. Fila superior: modelo entrenado desde cero. Fila inferior: modelo con ajuste fino a partir de pesos preentrenados en ImageNet.	65

ÍNDICE DE TABLAS

3.1	Comparación entre las descomposiciones CP, Tucker y TT/MPS.	35
3.2	Comparativa sintética de topologías relevantes de Tensor Networks. Los conteos de parámetros se expresan a orden de magnitud; en las filas de TTN y MERA se omite por simplicidad el factor asociado a la dimensión local d , que sí se indica explícitamente en MPS/TT y PEPS.	41
5.1	Métricas principales del mejor modelo tensorial sobre ChemEq25.	58
5.2	Comparación entre el modelo tensorial propuesto y ResNet50 sobre los mismos recortes de ChemEq25.	62

Capítulo 1. INTRODUCCIÓN

La clasificación de imágenes es una de las tareas centrales del *machine learning*. Consiste en asignar a una imagen una etiqueta entre un conjunto finito de clases, a partir de patrones visuales que pueden depender, de forma simultánea, del color, la forma, la textura, la escala, la iluminación y el contexto. En los últimos años, los modelos de *deep learning* han demostrado una gran capacidad para resolver este tipo de problemas, pero también han puesto de manifiesto un problema importante. Los modelos más expresivos suelen requerir un número elevado de parámetros, grandes cantidades de datos y un coste computacional considerable.

Este TFG parte de esa problemática y estudia una familia de modelos alternativa, inspirada en herramientas procedentes de la física cuántica y del álgebra multilineal; las Tensor Networks (TN). La idea general es representar objetos de dimensión muy alta mediante la contracción de tensores más pequeños, de forma que el número de parámetros crezca de manera controlada.

La motivación física aparece de manera natural. En mecánica cuántica, describir exactamente un sistema de muchas partículas exige manejar espacios de Hilbert cuya dimensión crece exponencialmente con el número de componentes. Las TN surgieron precisamente de una idea que sirvió para representar de forma eficiente estados cuánticos relevantes sin almacenar explícitamente todos sus coeficientes. En *machine learning* aparece un problema análogo. Una imagen contiene muchas variables, y una función de decisión general sobre todas ellas vive, en principio, en un espacio de dimensión enorme. La pregunta que guía este trabajo es, si las estructuras tensoriales pueden aprovechar regularidades de los datos visuales, para construir clasificadores entrenables y analizables.

1.1 Estado del Arte

La literatura reciente sobre las Tensor Networks aplicadas a *machine learning* puede organizarse en tres líneas principales. La primera utiliza descomposiciones tensoriales para reducir dimensionalidad o comprimir modelos ya existentes. En esta línea, las TN se emplean como herramienta de factorización de pesos, capas densas, convoluciones o representaciones intermedias, con el objetivo de reducir memoria y coste computacional [6, 5]. La segunda línea utiliza las TN como modelos de aprendizaje en sí mismos, es decir, como clasificadores o generadores entrenables. La tercera, más reciente, estudia arquitecturas híbridas que combinan ideas tensoriales con mecanismos habituales de *deep learning*, como normalización, *patch embeddings*, capas convolucionales ligeras o entrenamiento extremo a extremo con *backpropagation* [5].

Un punto de partida fundamental para la aplicación de TN a clasificación supervisada

es el trabajo de Stoudenmire y Schwab [7]. En dicho trabajo, una imagen se transforma mediante un mapa local de características y el clasificador se interpreta como una función lineal en un espacio de características de dimensión exponencial, cuyo tensor de pesos se representa de forma compacta mediante una Matrix Product State (MPS). El resultado mostró que una MPS podía alcanzar un error de test inferior al 1% en MNIST. Este resultado fue importante porque demostró que las redes tensoriales podían actuar como modelos supervisados, no solo como herramientas en física cuántica o como herramientas de compresión. Sin embargo, también marcó una limitación que sigue siendo relevante; la MPS impone una estructura lineal sobre los datos. En imágenes sencillas y pequeñas, como MNIST, esta linealización puede ser suficiente; sin embargo, en imágenes naturales, en color o de mayor resolución, la pérdida de localidad espacial se vuelve mucho más problemática.

Una respuesta natural a esa limitación consiste en utilizar topologías que respeten mejor la estructura bidimensional de las imágenes. Los Projected Entangled Pair States (PEPS) son especialmente atractivos porque organizan los tensores sobre una rejilla 2D, más cercana a la geometría original de una imagen. Cheng, Wang y Zhang [8] aplicaron PEPS a clasificación supervisada en MNIST y Fashion-MNIST, mostrando que una red tensorial bidimensional puede superar a topologías tipo árbol en esos escenarios y funcionar con un número reducido de parámetros. No obstante, PEPS introduce una dificultad importante; la contracción exacta de redes tensoriales bidimensionales es computacionalmente intratable en el caso general [9]. Por ello, aunque PEPS es conceptualmente muy adecuado para imágenes, su uso práctico suele requerir contracciones aproximadas, restricciones en las dimensiones de enlace o implementaciones cuidadosas.

Entre la MPS lineal y PEPS bidimensional aparecen las Tree Tensor Networks (TTN), que organizan la información de manera jerárquica. Esta familia resulta interesante porque reduce la distancia efectiva entre regiones alejadas de la entrada y permite agregar información de forma progresiva. En comparación con una cadena MPS, una TTN puede capturar interacciones de mayor alcance con menos pasos de propagación. Aun así, al no tener ciclos, las interacciones entre ramas distintas quedan mediadas por ancestros comunes, lo que limita la comunicación directa entre regiones separadas. Por tanto, la TTN representa un compromiso entre eficiencia de contracción, estructura jerárquica y capacidad expresiva [5, 10].

La tendencia más reciente no se limita a elegir una topología física clásica, sino a re-interpretar las TN desde el punto de vista de las arquitecturas modernas. Un ejemplo especialmente relevante es Deep Tree Tensor Networks (DTTN), que propone apilar módulos de interacción antisimétrica (AIM) para capturar interacciones multiplicativas de orden creciente y mantener, al mismo tiempo, una estructura desplegable como red tensorial de tipo árbol [11]. Este trabajo es importante para el presente TFG por dos motivos. Primero, identifica de forma explícita que las MPS tradicionales han tenido dificultades para escalar a reconocimiento de imágenes naturales cuando se usan como núcleo principal del modelo. Segundo, conecta las TN con redes polinómicas y multilineales modernas,

mostrando que la investigación actual se mueve hacia arquitecturas híbridas, entrenables con herramientas estándar y mejor adaptadas a datos visuales complejos.

Desde el punto de vista de implementación, también ha habido una evolución clara. Las primeras propuestas de aprendizaje con MPS estaban muy ligadas a algoritmos inspirados en DMRG y a código de investigación específico. Actualmente existen librerías que buscan integrar TN dentro de ecosistemas de *deep learning*. TensorKrowch, por ejemplo, está construido sobre PyTorch y permite definir redes tensoriales entrenables, conectarlas con otros módulos y aprovechar diferenciación automática [12]. Esta disponibilidad de herramientas no elimina las dificultades del problema; siguen siendo decisiones críticas la elección del *embedding*, el orden de las posiciones de entrada de datos, la topología, la dimensión de enlace, la ruta de contracción y la estabilidad del entrenamiento.

En conjunto, el estado del arte muestra que las TN son una línea prometedora para construir modelos compactos, estructurados e interpretables, pero no constituyen todavía una alternativa universal a las arquitecturas dominantes de *machine learning*, como CNNs o Transformers. Su ventaja es más clara cuando se busca controlar el número de parámetros, introducir sesgos estructurales, modelar interacciones multilineales o estudiar arquitecturas inspiradas en física cuántica. Su mayor debilidad práctica aparece al trasladar resultados de datasets sencillos a imágenes reales de mayor resolución, donde la representación espacial y el coste de contracción se vuelven determinantes. Este TFG se sitúa precisamente en ese punto. Parte de modelos consolidados como MPS, estudia alternativas jerárquicas tipo TTN y explora mecanismos híbridos inspirados en trabajos recientes, sin asumir que una red tensorial vaya a superar de forma automática a una *deep learning* convencional.

1.2 Planteamiento del Problema

El problema que aborda este TFG puede formularse de la siguiente manera. Estudiar si una arquitectura basada en Tensor Networks puede utilizarse de forma práctica para resolver una tarea de clasificación de imágenes, manteniendo un número controlado de parámetros y una estructura interpretable, sin depender exclusivamente de arquitecturas profundas convencionales.

Esta formulación general contiene varios retos. El primero es la representación de la imagen. Una imagen digital no es, para una red tensorial, una entrada natural en forma de matriz o mapa de píxeles, sino un objeto que debe convertirse previamente en una secuencia de vectores locales. En datasets simples como el MNIST, esta conversión puede hacerse píxel a píxel. Sin embargo, cuando las imágenes son RGB, tienen mayor resolución y contienen objetos reales con forma, orientación y textura, tratar cada píxel como un sitio tensorial produce secuencias demasiado largas y poco manejables. Por tanto, una parte central del problema consiste en diseñar un *embedding* que reduzca la imagen a una representación compatible con la TN, pero que no destruya por completo la información espacial relevante.

El segundo problema es la elección de la topología tensorial. Una MPS resulta atractiva por su simplicidad y por la madurez de sus algoritmos, pero impone una geometría lineal que no se ajusta de forma natural a una imagen bidimensional. Una PEPS conserva mejor la estructura 2D, pero su contracción exacta es mucho más costosa. Una TTN ofrece una solución intermedia; organiza la información de forma jerárquica, reduce la distancia efectiva entre posiciones de entrada y mantiene una contracción más manejable. Por ello, el proyecto plantea estudiar arquitecturas tensoriales que puedan entrenarse en la práctica y que sean razonables para imágenes reales, aunque no reproduzcan literalmente todos los modelos más complejos de la literatura.

El tercer problema es de implementación. Pasar de una definición matemática de una Tensor Network a un modelo entrenable requiere un nivel de computación elevado. Es necesario construir nodos tensoriales con parámetros, conectar correctamente sus índices, definir una ruta de contracción, integrar la red con PyTorch, calcular gradientes, entrenar por *batches* y registrar métricas. Además, las capas tensoriales deben encajar con el resto del sistema de clasificación, carga de datos, transformaciones, función de pérdida, optimizador, *scheduler*, *checkpoints* y evaluación.

El cuarto problema es experimental. No basta con que el modelo pueda ejecutarse y disminuir la pérdida de entrenamiento; hay que evaluar si aprende patrones visuales que generalicen al conjunto de validación y test. Para ello, el trabajo no puede limitarse a una única métrica final. Es necesario analizar precisión, *macro F1*, curvas de entrenamiento, matrices de confusión, comportamiento por clase y posibles señales de *overfitting* o sobreconfianza. Esta evaluación es especialmente importante porque las TN pueden ser eficientes en parámetros, pero esa eficiencia no garantiza por sí misma un buen rendimiento en clasificación de imágenes.

En la versión final del proyecto, el problema se concreta sobre ChemEq25, un dataset de imágenes de material de laboratorio con 25 clases. Las imágenes originales contienen varios objetos, por lo que el problema se reformula como clasificación de instancias; a partir de las anotaciones se generan recortes individuales, y cada recorte debe clasificarse en una única clase. Así, el sistema completo debe resolver la cadena:

`imagen anotada → crop de objeto → embedding → TN → clase predicha.`

En este flujo, la TN se emplea específicamente en la etapa de clasificación de los objetos recortados. El resto de tareas como la extracción, el preprocesamiento y la preparación de las imágenes siguen un flujo convencional de visión artificial implementado mediante PyTorch.

La pregunta principal del trabajo no es si las Tensor Networks sustituyen de manera general a las CNNs o a los Transformers, sino si es posible construir, entrenar y evaluar un clasificador de imágenes basado en TN que funcione de extremo a extremo sobre un problema visual realista, y qué limitaciones aparecen durante ese proceso. A partir de esta pregunta principal se derivan varias cuestiones:

- ¿Qué tipo de representación de imagen permite alimentar una tensor network sin producir secuencias inviables?
- ¿Qué limitaciones aparecen al usar una MPS como punto de partida para imágenes de mayor resolución?
- ¿Puede una estructura jerárquica tipo TTN ofrecer un compromiso razonable entre expresividad y coste computacional?
- ¿Qué aporta combinar contracciones tensoriales con componentes habituales de *deep learning*, como normalización, capas lineales o un *front-end* espacial?
- ¿Qué rendimiento y qué patrones de error se observan al entrenar el modelo sobre un dataset real de clasificación de objetos?

Por tanto, el planteamiento del problema consiste en analizar hasta qué punto las TN son adecuadas para clasificación de imágenes más allá de ejemplos pequeños y controlados.

1.3 Planificación del TFG

- **Semanas 1–3: Introducción y fundamentos de Tensor Networks**

Durante las primeras semanas se adquirirán los conocimientos fundamentales y teóricos sobre Tensor Networks.

- Estudio de la notación y representaciones básicas de Tensor Networks mediante material audiovisual.
- Lectura y análisis de artículos científicos relevantes, con especial atención a modelos tensoriales aplicados a clasificación de imágenes.
- Estudio de apuntes y material teórico relacionado con Tensor Networks y su conexión con la física de sistemas cuánticos.

- **Semanas 4–6: Estudio de modelos MPS para clasificación de imágenes**

En esta etapa se profundizará en el uso de Matrix Product States como modelos de clasificación de imágenes:

- Análisis del funcionamiento de clasificadores basados en MPS y estudio de sus fuentes de error.
- Revisión de implementaciones existentes y comprensión de los procedimientos de entrenamiento y evaluación.
- Búsqueda y selección de conjuntos de datos adecuados en plataformas como Kaggle o Hugging Face.

- **Semanas 7–8: Formación en herramientas computacionales**

Durante este periodo se adquirirá el conocimiento práctico necesario para el desarrollo del proyecto:

PLANIFICACION DEL TFG

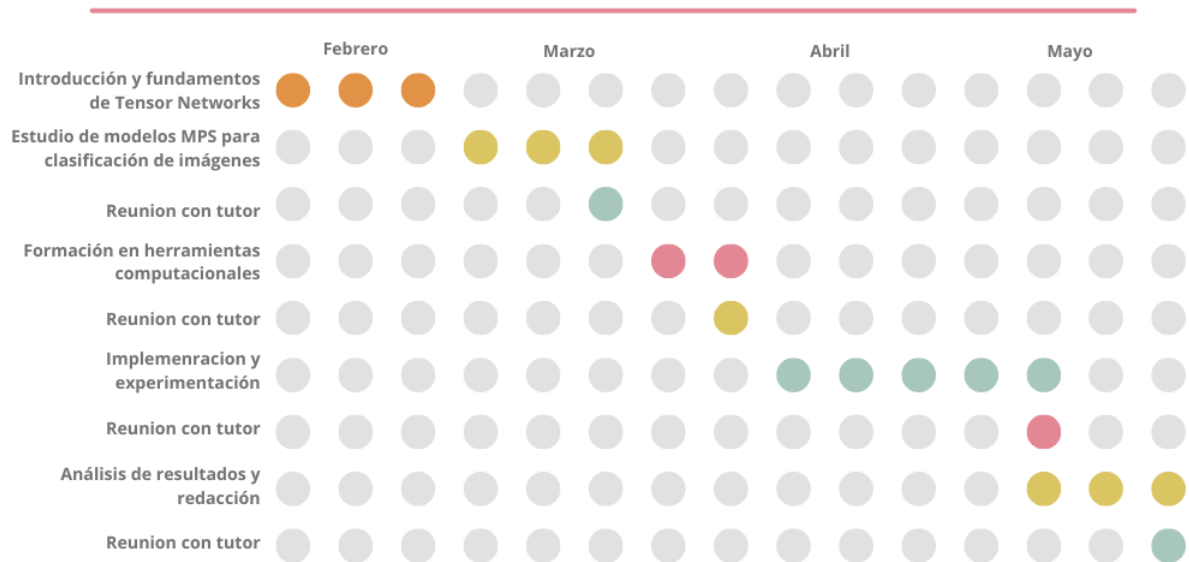


Figura 1.1: Planificación por semanas del TFG, cada círculo representa una semana teniendo febrero 3 semanas, marzo 4 semanas, abril cuatro semanas y por último mayo tendrá otras cuatro semanas. De esta manera se cumplen entre 500 y 600 horas dedicadas al TFG.

- Aprendizaje de comandos básicos de Bash y trabajo en entornos de servidor.
- Uso de GitHub para el control de versiones y la organización del código.
- Estudio de buenas prácticas de organización de proyectos y documentación.
- Aprendizaje en profundidad de PyTorch para la implementación de modelos de inteligencia artificial.

■ **Semanas 9–13: Implementación y experimentación**

En esta fase se llevará a cabo la implementación y entrenamiento de los modelos seleccionados:

- Implementación de modelos basados en Tensor Networks para clasificación de imágenes.
- Entrenamiento de los modelos utilizando recursos de computación de altas prestaciones (HPC).
- Evaluación del rendimiento de los modelos y comparación con modelos de inteligencia artificial convencionales.

■ **Semanas 13–15: Análisis de resultados y redacción**

En la etapa final se realizará el análisis de los resultados obtenidos y la documentación del trabajo:

- Análisis crítico de los resultados y discusión de las conclusiones.
- Redacción de la memoria del Trabajo Fin de Grado.
- Organización final del repositorio de código y preparación de la entrega.

1.4 Recursos requeridos

A continuación se enumeran los recursos técnicos utilizados para la ejecución del trabajo:

- Recursos computacionales:
 - Servidor HPC Lorca de la Universidad Europea de Madrid: para el entrenamiento de los modelos con recursos de computación de altas prestaciones.
 - Ordenador personal: para el desarrollo, pruebas locales y escritura de la memoria.
- Herramientas de software
 - Python (lenguaje de programación principal).
 - PyTorch (framework de deep learning).
 - TensorKrowch (librería para implementación de Tensor Networks).
 - NumPy, SciPy, Matplotlib, Seaborn (librerías científicas y de visualización).

- GitHub (control de versiones y gestión del repositorio de código).
- Bash / Linux (gestión del entorno HPC y automatización de trabajos).
- Docker / entornos virtuales (gestión de dependencias y reproducibilidad).
- Recursos bibliográficos y de datos:
 - Artículos científicos de referencia (véase sección de Referencias).
 - Conjuntos de datos públicos: MNIST, Flowers102 dataset y ChemEq25 dataset [\[13\]](#).

Capítulo 2. OBJETIVOS

2.1 Objetivos generales

El objetivo general de este Trabajo de Fin de Grado es estudiar la aplicación de las *Tensor Networks* como una alternativa dentro de modelos de *deep learning* para clasificación de imágenes. En particular, se busca diseñar, implementar y evaluar una arquitectura basada en redes tensoriales aplicada a un problema realista de clasificación de imágenes de material de laboratorio.

El propósito del trabajo no es demostrar que las redes tensoriales sustituyan de forma general a las arquitecturas convencionales de visión artificial. Más bien se trata de analizar hasta qué punto pueden utilizarse en este tipo de tareas, qué decisiones de diseño resultan más importantes y qué limitaciones aparecen durante su implementación y entrenamiento. Por ello, el modelo se compara con una arquitectura estándar de referencia, ResNet50, con el fin de contextualizar su rendimiento dentro de un marco experimental más amplio.

De forma complementaria, el proyecto persigue adquirir experiencia práctica en el uso de herramientas computacionales propias del ámbito de la investigación en inteligencia artificial. Incluyendo el manejo de entornos de entrenamiento, el uso de librerías como PyTorch y TensorKrowch, la ejecución de experimentos en recursos de computación de altas prestaciones y la documentación reproducible del código científico.

2.2 Objetivos específicos

1. Estudiar los fundamentos de las *Tensor Networks* y su posible aplicación como modelos entrenables en problemas de *machine learning*, prestando especial atención a arquitecturas como las Matrix Product States y las Tree Tensor Networks.
2. Revisar trabajos previos en los que se hayan aplicado redes tensoriales a tareas de aprendizaje automático y clasificación de imágenes, identificando sus motivaciones, ventajas potenciales y principales limitaciones.
3. Diseñar un *pipeline* computacional para clasificación de imágenes que integre de forma ordenada la carga de datos, el preprocesamiento, la definición del modelo, el entrenamiento, la evaluación y el registro de resultados.
4. Diseñar e implementar una arquitectura basada en redes tensoriales.
5. Entrenar y evaluar los modelos desarrollados utilizando recursos computacionales adecuados, documentando las configuraciones empleadas y los resultados obtenidos de forma reproducible.

6. Comparar la arquitectura tensorial propuesta con una arquitectura convolucional de referencia, ResNet50, considerando distintos esquemas de entrenamiento (entrenamiento desde cero y ajuste fino de pesos preentrenados).
7. Evaluar los modelos mediante métricas de clasificación adecuadas, como la precisión global, la precisión top-5, el F1 macro, las curvas de entrenamiento y el análisis de errores por clase.

2.3 Objetivos que quedan fuera del proyecto

- Realizar una comparación exhaustiva con todas las arquitecturas modernas de visión artificial. El trabajo incluye una comparación de referencia con ResNet50, pero no pretende cubrir de forma sistemática modelos como *Vision Transformers*, *EfficientNet* u otras arquitecturas avanzadas.
- Resolver otros problemas del campo de la inteligencia artificial, como detección de objetos, segmentación semántica o generación de imágenes.
- Realizar una comparación exhaustiva frente a todas las arquitecturas convencionales de *deep learning*, o evaluar los modelos en un gran número de benchmarks adicionales.
- Ejecutar los modelos en hardware cuántico real, más allá de enfoques cuántico-inspirados ejecutados en sistemas clásicos.
- Desarrollar aplicaciones orientadas a entornos industriales, sistemas en producción o herramientas de inferencia en tiempo real a partir de los modelos estudiados.

2.4 Beneficios del proyecto

Este proyecto contribuye a ampliar el conocimiento sobre las Tensor Networks como herramienta de aprendizaje automático. Aunque el fundamento teórico de estas arquitecturas está bien establecido en el ámbito de la física cuántica y el álgebra multilineal, su comportamiento práctico en tareas de clasificación de imágenes sigue siendo un área activa de investigación. El trabajo permite analizar qué dificultades aparecen al trasladar estas ideas a un pipeline real de *deep learning*.

Desde el punto de vista computacional, el proyecto permite explorar modelos estructurados en los que la capacidad se controla mediante decisiones como la topología de la red, la dimensión de enlace o la representación de entrada. Esto ofrece una perspectiva complementaria a la del *deep learning* convencional, centrada en introducir sesgos estructurales y operaciones tensoriales en el proceso de aprendizaje.

La realización de este proyecto supone también una experiencia práctica en investigación aplicada. Comprende la revisión crítica de literatura científica, el diseño e implementación de experimentos reproducibles, el uso de infraestructuras de entrenamiento y el aprendizaje

de herramientas y flujos de trabajo propios del ámbito profesional de la inteligencia artificial. Estos conocimientos son transferibles a entornos académicos y laborales relacionados con la física, la computación científica y el aprendizaje automático.

Capítulo 3. MARCO TEÓRICO

3.1 Deep learning y clasificación de imágenes

Antes de explicar el concepto de *deep learning* es importante entender qué es una red neuronal. Una red neuronal es una estructura computacional formada por un conjunto de unidades de procesamiento simples, denominados neuronas. Estas están interconectadas mediante pesos sinápticos ajustables, llamados en inglés *weights* y *biases*. Cada neurona recibe una serie de entradas, a las cuales llamaremos *inputs*; se calcula una suma ponderada de las mismas y se les aplica una función de activación para producir una salida, la cual llamaremos *output*. El comportamiento de la red queda determinado por los valores de estos pesos, que son modificados durante el entrenamiento con el objetivo de minimizar una función de error entre los *outputs* y las respuestas esperadas.

Formalizando esta descripción, dada una entrada $\mathbf{x} = (x_1, x_2, \dots, x_n) \in \mathbb{R}^n$, la operación de una única neurona se expresa como:

$$y = \sigma \left(\sum_{i=1}^n w_i x_i + b \right) = \sigma(\mathbf{w}^\top \cdot \mathbf{x} + b) \quad (3.1)$$

donde $\mathbf{w} = (w_1, w_2, \dots, w_n)$ es el vector de pesos sinápticos, b es el sesgo (*bias*), y $\sigma : \mathbb{R} \rightarrow \mathbb{R}$ es la función de activación. El argumento de σ es una transformación afín de la entrada, es decir, un producto escalar seguido de una traslación. La no linealidad introducida por σ es esencial, pues sin ella la composición de múltiples capas lineales se reduciría a una única transformación lineal, independientemente de la profundidad de la red [14].

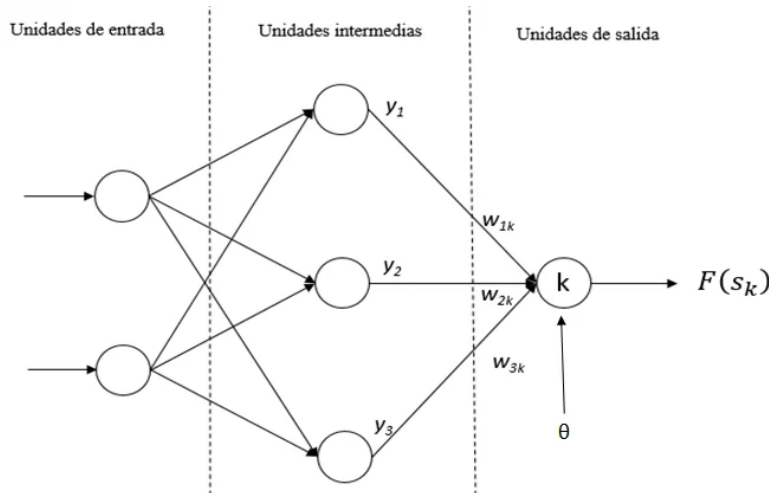


Figura 3.1: Esquema de una red neuronal simple. [1]

Las funciones de activación más utilizadas en la práctica incluyen la sigmoide $\sigma(z) = 1/(1+e^{-z})$, la tangente hiperbólica $\sigma(z) = \tanh(z)$, y la unidad lineal rectificada (ReLU) $\sigma(z) = \max(0, z)$. La elección de la función de activación tiene consecuencias importantes sobre la dinámica del entrenamiento. Funciones como la sigmoide pueden producir gradientes que se desvanecen en redes profundas, mientras que la ReLU mitiga parcialmente este problema [14].

Una capa completa de la red consiste en m neuronas que operan en paralelo sobre la misma entrada, produciendo un vector de salida $\mathbf{y} \in \mathbb{R}^m$:

$$\mathbf{y} = \sigma(\mathbf{W}\mathbf{x} + \mathbf{b}) \quad (3.2)$$

donde $\mathbf{W} \in \mathbb{R}^{m \times n}$ es la matriz de pesos de la capa y $\mathbf{b} \in \mathbb{R}^m$ el vector de sesgos.

Llegamos a la definición de *deep learning*, que no es más que el uso de una red neuronal con muchas capas intermedias, lo que se conoce como red profunda. La expresión matemática que define una red profunda es una extensión de la Ecuación 3.2 pero para L capas:

$$f(\mathbf{x}) = \sigma_L(\mathbf{W}_L \cdots \sigma_2(\mathbf{W}_2 \sigma_1(\mathbf{W}_1 \mathbf{x} + \mathbf{b}_1) + \mathbf{b}_2) \cdots + \mathbf{b}_L) \quad (3.3)$$

Los parámetros entrenables del modelo son el conjunto $\{\mathbf{W}_1, \mathbf{b}_1, \mathbf{W}_2, \mathbf{b}_2, \dots, \mathbf{W}_L, \mathbf{b}_L\}$, cuyo número total crece con la anchura y la profundidad de la red.

El entrenamiento de una red neuronal se formula como un problema de optimización. Dado un conjunto de entrenamiento $\{(\mathbf{x}_n, y_n)\}_{n=1}^{N_T}$, se define una *función de coste* (o función de pérdida) C que mide la discrepancia entre las predicciones del modelo $f(\mathbf{x}_n)$ y las etiquetas verdaderas y_n . El objetivo es encontrar los parámetros $\theta = \{\mathbf{W}_l, \mathbf{b}_l\}$ que minimizan C :

$$\theta^* = \arg \min_{\theta} C(\theta) = \arg \min_{\theta} \frac{1}{N_T} \sum_{n=1}^{N_T} \mathcal{L}(f(\mathbf{x}_n; \theta), y_n) \quad (3.4)$$

donde $\mathcal{L}$ es una función de pérdida por muestra (por ejemplo, el error cuadrático medio o la entropía cruzada). Este es un problema de minimización en un espacio de alta dimensión, que se resuelve mediante variantes del *descenso de gradiente* [14].

El algoritmo fundamental para calcular los gradientes necesarios es la *backpropagation*, que aplica sistemáticamente la regla de la cadena del cálculo diferencial para propagar el error desde la capa de salida hacia las capas inferiores. Para un parámetro θ cualquiera de la red, el gradiente se calcula como:

$$\frac{\partial C}{\partial \theta} = \frac{\partial C}{\partial \mathbf{y}_L} \cdot \frac{\partial \mathbf{y}_L}{\partial \mathbf{y}_{L-1}} \cdots \frac{\partial \mathbf{y}_{l+1}}{\partial \mathbf{y}_l} \cdot \frac{\partial \mathbf{y}_l}{\partial \theta} \quad (3.5)$$

donde $\mathbf{y}_l$ es la salida de la capa l -ésima y cada factor $\partial \mathbf{y}_{l+1} / \partial \mathbf{y}_l$ es una matriz jacobiana, por lo que el orden de los productos es relevante. La actualización de los parámetros sigue

entonces la regla $\theta \leftarrow \theta - \eta \partial C / \partial \theta$, donde $\eta > 0$ es la tasa de aprendizaje.

La diferencia fundamental entre las redes clásicas y las redes *deep* radica en su capacidad de representación. Una red con una única capa oculta puede, en teoría, aproximar cualquier función continua con la suficiente anchura, pero resulta ineficiente para capturar estructuras jerárquicas, como las que están presentes en las imágenes o en el lenguaje natural [15, 16, 6]. Las redes profundas, en cambio, aprenden representaciones en múltiples niveles de abstracción. Las capas más bajas detectan características primitivas como bordes o frecuencias, mientras que las superiores combinan estas primitivas para reconocer objetos complejos o conceptos semánticos [5].

En clasificación de imágenes, las redes convolucionales han sido especialmente relevantes porque incorporan localidad espacial y pesos compartidos. Estas dos propiedades son adecuadas para datos visuales, porque los píxeles vecinos tienden a estar relacionados, y los mismos patrones pueden aparecer en distintas regiones de la imagen. La Figura 3.2 muestra algunas topologías básicas de redes neuronales.

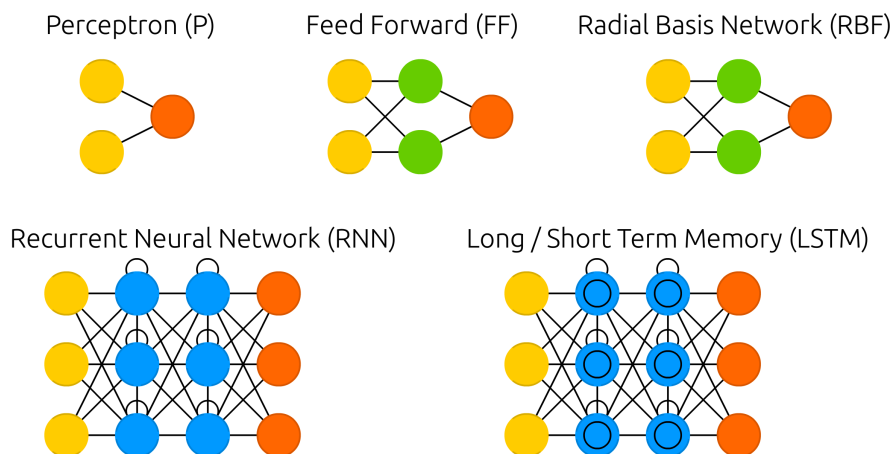


Figura 3.2: Esquema de topologías básicas de redes neuronales, desde el perceptrón hasta arquitecturas con capas ocultas y conexiones recurrentes [2].

El éxito de estas arquitecturas no elimina sus limitaciones. Muchos modelos modernos alcanzan buenos resultados con un número elevado de parámetros y con un coste computacional considerable. Este TFG se sitúa en ese punto. Estudia si las TN pueden proporcionar modelos de clasificación estructurados, capaces de representar funciones complejas mediante factorizaciones tensoriales controladas.

3.2 Tensores

Antes de profundizar en el mundo de las TN es conveniente sentar primero las bases. Lo primero es entender qué es un tensor. Un tensor lo podemos ver definido de diferentes formas dependiendo del campo en el que estemos trabajando.

Desde un punto de vista computacional, los tensores son *arrays* multidimensionales que generalizan de forma natural los conceptos de escalar, vector y matriz a dimensiones arbitrariamente superiores. Desde un punto de vista físico-matemático, los tensores son objetos matemáticos de orden superior, que representan relaciones multidimensionales entre distintas variables o grados de libertad. Dicho de otra forma, un tensor de orden N es un objeto cuyos elementos tienen N índices y están asociados a N vectores base.

$$R^{I_1 \times I_2 \times \dots \times I_N}$$

donde N es el número de modos y las longitudes $I_1, \dots, I_N$ representan las dimensiones de cada modo. [5]

Bajo esta definición, los escalares son tensores de orden cero, los vectores son tensores de orden 1, y las matrices son tensores de orden 2; todo caso superior constituye propiamente lo que en la literatura se denomina, tensor de alto orden. En la notación convencional, se distinguen los diferentes objetos según su orden: las letras minúsculas en cursiva (a) denotan escalares, las letras minúsculas en negrita ($\mathbf{a}$) denotan vectores, las letras mayúsculas en negrita ($\mathbf{A}$) denotan matrices, y las letras mayúsculas en tipografía cursiva en negrita ($\mathcal{A}$) denotan tensores de orden superior. [5]

La intuición física detrás de los tensores resulta natural cuando se considera que muchos fenómenos del mundo real son inherentemente multidimensionales. Una imagen en color, por ejemplo, es un tensor de orden tres: altura, anchura y canales cromáticos. Los datos de resonancia magnética funcional (fMRI) constituyen tensores de orden cuatro. En mecánica cuántica, las funciones de onda de sistemas de muchos cuerpos son, igualmente, tensores de alto orden [5].

El concepto de tensor tiene raíces en el siglo XIX, ligado al desarrollo del cálculo diferencial absoluto por Ricci-Curbastro y Levi-Civita, aunque fue la formulación de la relatividad general por Einstein en 1915 la que consagró los tensores como herramienta fundamental de la física matemática. La descripción matemática de un tensor es la siguiente:

$$\mathcal{T} = \sum_{i_1, i_2, \dots, i_n} T_{i_1 i_2 \dots i_n} \hat{b}_{i_1} \hat{b}_{i_2} \dots \hat{b}_{i_n} \quad (3.6)$$

La operación central en el álgebra tensorial es la contracción tensorial. Dados dos tensores $\mathcal{T}$ y $\mathcal{R}$, de orden n y m respectivamente, con elementos $T_{i_1 i_2 \dots i_n}$ y $R_{j_1 j_2 \dots j_m}$ y en la misma base ortonormal $\hat{b}_i$, diremos que el tensor $\mathcal{C}$ es la contracción de ambos tensores por sus últimos q índices cuando:

$$\mathcal{C} = \sum_{i_1, i_2, \dots, i_{n-q}, j_1, j_2, \dots, j_{m-q}} C_{i_1 i_2 \dots i_{n-q}, j_1 j_2 \dots j_{m-q}} \hat{b}_{i_1} \hat{b}_{i_2} \dots \hat{b}_{i_{n-q}} \hat{b}_{j_1} \hat{b}_{j_2} \dots \hat{b}_{j_{m-q}} \quad (3.7)$$

siendo sus elementos

$$C_{i_1 i_2 \dots i_{n-q}, j_1 j_2 \dots j_{m-q}} = \sum_{k_1, k_2, \dots, k_q} T_{i_1 i_2 \dots i_{n-q}, k_1 k_2 \dots k_q} R_{j_1 j_2 \dots j_{m-q}, k_1 k_2 \dots k_q} \quad (3.8)$$

De esta definición podemos sacar que la contracción tensorial es la operación mediante la cual dos tensores se contraen en un único tensor a lo largo de sus pares de índices asociados. Existen formas diferentes de contracciones, algunas implican más de dos tensores o un índice repetido en más de dos tensores o incluso repetidos en el mismo tensor. El caso más familiar de contracción tensorial es la multiplicación matricial:

$$\mathbf{AB} = \sum_k A_{ik} B_{kj} \hat{b}_i \hat{b}_j$$

o el producto escalar entre dos vectores:

$$\mathbf{v} \cdot \mathbf{w} = \sum_i v_i w_i$$

Entre las operaciones tensoriales fundamentales cabe destacar la traza de una matriz:

$$Tr(\mathbf{A}) = \sum_i A_{ii} \quad (3.9)$$

Esto sería la contracción de una matriz consigo misma a través de sus índices repetidos. Debido a esta operación podemos ver que, un tensor también sirve como mapeo entre dos espacios vectoriales. Esto es un tensor, que recibe otro tensor y devuelve un tercer tensor. Por cada par de índices contraído estamos mezclando componentes de los tensores de entrada, reduciendo en 2 el orden total. Los índices que no han sido contraídos, en cambio, no desaparecen, sino que sobreviven como índices libres del tensor de salida. Así, al contraer un tensor de orden n con uno de orden m a lo largo de q pares de índices, el tensor resultante tiene orden $n + m - 2q$, en consonancia con la [Ecuación 3.8](#).

3.3 Notación diagramática de Penrose y definición de Tensor Network

La representación algebraica de las contracciones tensoriales, aunque precisa, se vuelve rápidamente inmanejable a medida que aumenta el número de tensores y la complejidad de sus conexiones. Fue Roger Penrose [\[17\]](#) quien, a principios de la década de 1970, introdujo la notación diagramática para tensores, proporcionando una alternativa visual que transformó la manera en que se razona sobre estas estructuras. Los diagramas de redes tensoriales se utilizan hoy en día, en investigación, para describir algoritmos cuánticos y algoritmos de *machine learning* [\[5\]](#).

La idea central de la notación de Penrose es representar cada tensor como un nodo del

que emanan tantas aristas como índices tenga el tensor. Concretamente, un nodo con una sola arista representa un vector $\mathbf{a} \in \mathbb{R}^I$; un nodo con dos aristas representa una matriz $\mathbf{A} \in \mathbb{R}^{I \times J}$; y un nodo con tres aristas representa un tensor $\mathcal{A} \in \mathbb{R}^{I_1 \times I_2 \times I_3}$. Para indicar que un par de índices tensoriales está siendo contraído, ambas aristas se unen mediante una línea. Por ejemplo, la contracción de un tensor de orden uno con uno de orden dos es la multiplicación matriz-vector habitual. Las aristas que no se conectan a ningún otro tensor se denominan aristas libres, y el número de estas aristas en el diagrama resultante es precisamente el orden del tensor final. Dos nodos que no están unidos mediante aristas, representan el producto tensorial entre ellos.



Figura 3.3: Representación de una TN inventada. En esta representación se pueden identificar muchas de las operaciones descritas en esta sección. Elaboración propia usando la librería Tensor-Network-Editor, creada por Alejandro Mata Ali. [3]

La notación gráfica tensorial ofrece numerosas ventajas sobre la notación tradicional. En forma gráfica, los índices no necesitan nombres ni etiquetas; operaciones como el producto escalar, la traza tensorial y la contracción pueden expresarse sin notación adicional; y el orden del tensor resultante puede leerse simplemente contando el número de aristas libres. En particular, si en el diagrama no quedan aristas libres tras todas las contracciones, el resultado es necesariamente un escalar. Esta propiedad convierte los diagramas en una herramienta de verificación inmediata de la consistencia dimensional de operaciones complejas [7]. Sobre esta base visual puede introducirse ya, de manera natural, el concepto de *Tensor Network (TN)*.

Una TN es, en esencia, una representación gráfica de una ecuación multilineal entre diferentes tensores. Esta representación se apoya en la notación diagramática de Penrose, que con el tiempo se ha ido adaptando a las necesidades de disciplinas como la física, las matemáticas o la ciencia computacional [18]. Puede verse un ejemplo en la Figura 3.4, donde aparece la TN que representa el 2-tensor T_{ip} cuyos elementos se obtienen mediante la operación de contracción

$$T_{ip} = \sum_{j,k,l,m,n,o} A_{ijkl} B_{jm} C_{kmn} D_{lo} E_{nop} \quad (3.10)$$

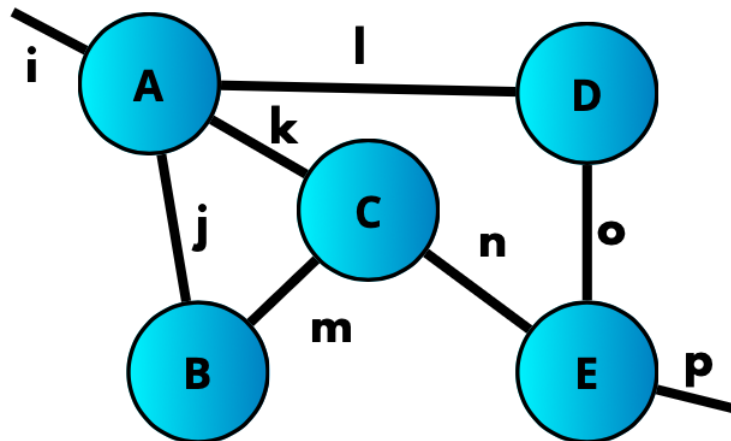


Figura 3.4: Ejemplo de representación diagramática para un tensor. Fuente: [4]

Desde un punto de vista computacional, una Tensor Network no debe entenderse únicamente como un dibujo, sino como una forma factorizada de representar un tensor de alto orden. En lugar de almacenar explícitamente todos los elementos del tensor completo, se almacena una colección de tensores de menor orden, conectados entre sí mediante índices compartidos. Cada tensor de la red suele denominarse también núcleo, bloque o tensor local, dependiendo del contexto [5].

En una red tensorial aparecen dos tipos de índices con papeles distintos. Los índices que quedan libres tras realizar las contracciones corresponden a los índices del tensor final representado por la red. En problemas de física cuántica suelen denominarse índices físicos, mientras que en aplicaciones de aprendizaje automático pueden interpretarse como posiciones de entrada, características o salidas del modelo. Por otro lado, los índices que conectan dos tensores internos se denominan índices virtuales o índices de enlace. Estos índices no aparecen en el tensor final, ya que se suman durante la contracción, pero controlan cuánta información puede transmitirse entre distintas partes de la red.

La dimensión de estos índices internos, denominada dimensión de enlace, es uno de los parámetros más importantes de una Tensor Network. Si la dimensión de enlace es pequeña, la red contiene pocos parámetros y la representación es muy compacta, pero también queda limitada su capacidad para representar correlaciones complejas. Si la dimensión de enlace aumenta, la red gana expresividad, aunque también crecen el número de parámetros y el coste computacional de las contracciones. Por tanto, la dimensión de enlace desempeña un papel análogo al de un hiperparámetro que regula el compromiso entre compactidad y capacidad de representación.

La operación fundamental en una Tensor Network es la contracción. Contraer una red significa sumar sobre todos los índices internos compartidos hasta obtener el tensor final, que queda determinado únicamente por sus índices libres. Aunque algebraicamente esta operación puede escribirse como una suma sobre muchos índices, la notación diagramática permite visualizarla de forma más intuitiva; las aristas conectadas se eliminan al con-

traerse, mientras que las aristas libres permanecen como índices del resultado. Además, el orden en el que se realizan las contracciones puede afectar de forma significativa al coste computacional, por lo que una misma red puede ser más o menos eficiente dependiendo de la estrategia de contracción empleada [5].

3.4 Tensor Networks y reducción de dimensionalidad

La motivación fundamental para el uso de las Tensor Networks es de naturaleza computacional. Un tensor de orden N cuyos índices toman cada uno d valores posee d^N componentes, un número que crece exponencialmente con el orden del tensor y que hace inviable su almacenamiento y manipulación directos incluso para valores moderados de N y d . Este fenómeno se conoce como *maldición de la dimensionalidad* [5].

El mismo problema aparece de forma natural en la física cuántica de muchos cuerpos, que es precisamente el contexto en el que surgieron las TN. Un estado cuántico general de N subsistemas con dimensión local d se escribe como

$$|\psi\rangle = \sum_{s_1, \dots, s_N} C_{s_1 \dots s_N} |s_1\rangle \otimes \dots \otimes |s_N\rangle, \quad (3.11)$$

donde el tensor de coeficientes $C_{s_1 \dots s_N}$ tiene de nuevo d^N componentes. Sin embargo, los estados físicamente relevantes rara vez ocupan todo ese espacio con la máxima complejidad posible, porque sus correlaciones y su entrelazamiento suelen estar limitados, lo que permite representarlos de forma mucho más compacta de lo que sugiere el conteo d^N [19].

La idea central de las Tensor Networks consiste en aprovechar esa estructura. En lugar de almacenar explícitamente el tensor completo, se representa como el resultado de contraer un conjunto de tensores de menor orden conectados entre sí mediante índices de enlace. Si las dimensiones de esos índices internos se mantienen acotadas, el número total de parámetros crece de forma polinómica e incluso lineal en algunas topologías, en lugar de exponencial con el número de modos [5, 7].

Esta reducción no es gratuita. Al fijar una topología concreta y unas dimensiones de enlace finitas se restringe la familia de tensores que pueden representarse de forma eficiente. Por ello, una Tensor Network debe entenderse como una aproximación estructurada; solo resulta eficaz cuando la estructura de correlaciones del problema es compatible con la topología elegida. Esta distinción es especialmente importante en *machine learning*, donde la red tensorial no se limita a comprimir un tensor ya conocido, sino que se utiliza como una arquitectura entrenable cuyos parámetros se ajustan a partir de los datos.

Esta observación se traslada al *machine learning* mediante una analogía útil. Un modelo de clasificación no necesita representar una función arbitraria sobre todas las entradas posibles, sino una función que aproveche las regularidades de los datos. Las Tensor Networks imponen una estructura de bajo rango o de conectividad concreta sobre el tensor de pesos,

de modo que el número de parámetros permanece controlado siempre que las dimensiones internas, llamadas dimensiones de enlace, se mantengan acotadas [6, 5].

3.5 Descomposiciones tensoriales fundamentales

Para comprender cómo se construyen las representaciones mediante TN, conviene introducir primero algunas descomposiciones tensoriales básicas. Cada una puede entenderse como una forma distinta de imponer estructura sobre un tensor de alto orden. La descomposición CP lo expresa como una suma de términos separables; la de Tucker introduce un núcleo central que acopla varios factores; y el formato Tensor Train (TT), conocido en física como Matrix Product State (MPS), organiza la información en una cadena de tensores conectados por índices de enlace. Cada uno ofrece un compromiso diferente entre compacidad, expresividad y coste computacional, como se verá a lo largo de la sección.

La herramienta algebraica fundamental es la descomposición en valores singulares (SVD). Dada una matriz $\mathbf{A} \in \mathbb{R}^{M \times N}$, la SVD la factoriza como

$$\mathbf{A} = \mathbf{U} \mathbf{\Sigma} \mathbf{V}^\top, \quad (3.12)$$

donde $\mathbf{U}$ y $\mathbf{V}$ son matrices ortogonales, y $\mathbf{\Sigma}$ contiene los valores singulares $\sigma_1 \geq \sigma_2 \geq \dots \geq 0$. La importancia de la SVD en TN se debe a que permite obtener la mejor aproximación de bajo rango en norma de Frobenius. Al conservar solo los R valores singulares dominantes se obtiene la matriz de rango R que minimiza el error de reconstrucción, resultado conocido como teorema de Eckart–Young [5].

Una primera generalización del rango matricial a tensores de orden superior es la descomposición CP. Dado un tensor $\mathcal{X} \in \mathbb{R}^{I_1 \times \dots \times I_N}$, CP lo aproxima como suma de R tensores de rango 1:

$$\mathcal{X} \approx \sum_{r=1}^R \mathbf{a}_r^{(1)} \circ \mathbf{a}_r^{(2)} \circ \dots \circ \mathbf{a}_r^{(N)}, \quad (3.13)$$

donde $\circ$ denota el producto exterior y R es el rango CP. Esta representación es muy compacta, con $R \sum_{n=1}^N I_n$ parámetros, pero presenta dificultades prácticas; determinar el rango CP es un problema NP-difícil y los métodos de ajuste habituales no garantizan alcanzar el óptimo global [20, 5]. En notación diagramática, CP se representa mediante factores vectoriales conectados a un nodo central diagonal.

La descomposición de Tucker es más flexible. En lugar de una única suma de componentes de rango 1, introduce un tensor núcleo $\mathcal{G}$ que modela las interacciones entre los factores:

$$\mathcal{X}_{i_1 i_2 \dots i_N} \approx \sum_{r_1=1}^{R_1} \dots \sum_{r_N=1}^{R_N} \mathcal{G}_{r_1 r_2 \dots r_N} A_{i_1 r_1}^{(1)} A_{i_2 r_2}^{(2)} \dots A_{i_N r_N}^{(N)}. \quad (3.14)$$

Los valores $R_1, \dots, R_N$ son los rangos de Tucker y pueden ser distintos para cada modo. Si las matrices de factores son ortogonales, se obtiene la Higher-Order Singular Value Decomposition (HOSVD), una extensión natural de la SVD al caso tensorial [5]. CP puede verse como un caso particular de Tucker cuando el núcleo es superdiagonal y todos los rangos coinciden. Sin embargo, el núcleo de Tucker contiene $\prod_{n=1}^N R_n$ parámetros, por lo que el coste vuelve a crecer exponencialmente con el orden del tensor.

Esta limitación motiva formatos más escalables como Tensor Train (TT), conocido en física como Matrix Product State (MPS) [21, 19]. TT representa un tensor de orden N como una cadena de tensores locales $\mathcal{G}^{(n)} \in \mathbb{R}^{R_{n-1} \times I_n \times R_n}$ conectados mediante índices virtuales. Su número de parámetros es

$$\sum_{n=1}^N R_{n-1} I_n R_n,$$

que crece linealmente con N si los rangos se mantienen acotados. Además, la descomposición TT puede construirse mediante SVD sucesivas, lo que proporciona estabilidad numérica y permite controlar el error mediante el truncamiento de valores singulares [5, 21]. Desde la perspectiva cuántica, los rangos TT coinciden con los rangos de Schmidt de las biparticiones correspondientes, conectando esta construcción con la noción de entrelazamiento discutida anteriormente [19].

La Tabla 3.1 resume las diferencias principales entre los formatos anteriores.

Formato	Parámetros	Observaciones
CP	$R \sum_n I_n$	Muy compacta, pero el rango CP es difícil de determinar.
Tucker	$\prod_n R_n + \sum_n I_n R_n$	Flexible, pero el núcleo crece exponencialmente con el orden.
TT/MPS	$\sum_n R_{n-1} I_n R_n$	Escalado lineal para rangos fijos; construible mediante SVD sucesivas.

Tabla 3.1: Comparación entre las descomposiciones CP, Tucker y TT/MPS.

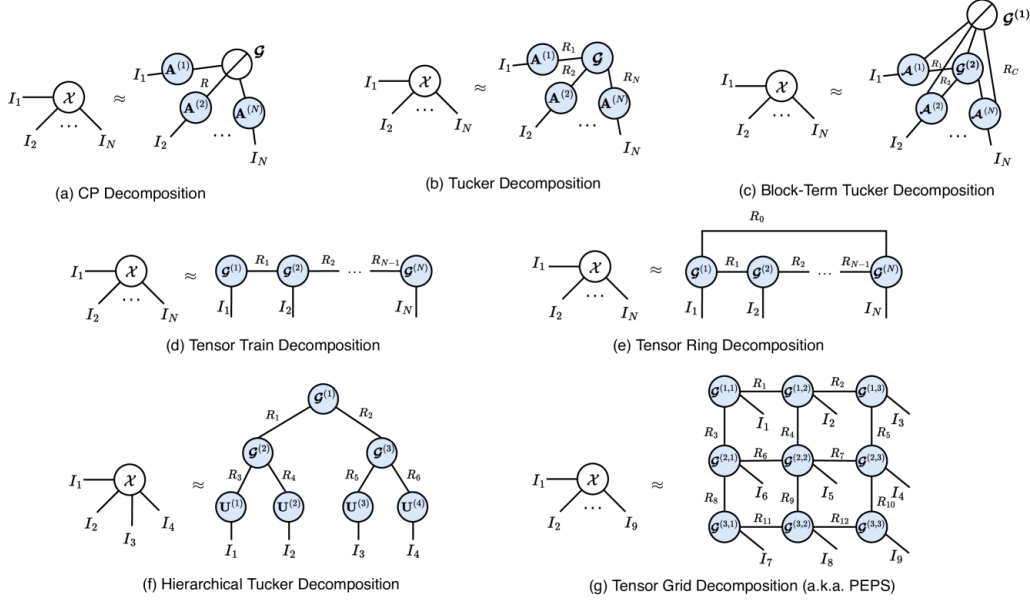


Figura 3.5: Diagramas TN de algunas descomposiciones tensoriales populares. (a) El formato CP descompone un tensor X en una suma de varios tensores de rango 1 $a_{:,r}^{(1)} \circ a_{:,r}^{(2)} \circ \dots \circ a_{:,r}^{(N)}$. (b) La descomposición de Tucker descompone un tensor X en un tensor núcleo G multiplicado por una matriz $A^{(n)}$ a lo largo del modo n -ésimo. (d) La descomposición TT descompone un tensor X en una multiplicación lineal de un conjunto de tensores núcleo de tercer orden $G^{(2)} \dots G^{(N-1)}$ y dos matrices $G^{(1)}$ y $G^{(N)}$. (g) La descomposición en malla tensorial, también conocida como PEPS, representa un tensor de alta dimensión como una malla bidimensional de tensores interconectados de menor rango, donde cada nodo se conecta con sus vecinos para capturar eficientemente correlaciones espaciales en sistemas con interacciones locales. Los paneles (c), (e) y (f) corresponden, respectivamente, a las descomposiciones en términos de bloque, Tensor Ring (TR) y jerárquica de Tucker (HT), no empleadas en este trabajo. [5]

3.6 Topologías principales de Tensor Networks

La topología de una Tensor Network determina qué correlaciones puede representar de forma eficiente y con qué coste computacional. En esta sección se detallan las topologías de TN más utilizadas y más relevantes en la literatura.

3.6.1 Matrix Product State / Tensor Train

El *Matrix Product State* (MPS), conocido en la literatura matemática como Tensor Train (TT), es la topología de TN más estudiada y mejor comprendida. Consiste en una cadena lineal de N tensores conectados secuencialmente mediante índices virtuales de dimensión D . Los tensores de los extremos tienen dos índices y los intermedios tres, un índice asociado a la posición de entrada y dos índices de enlace que conectan con los vecinos. Ambos nombres describen la misma topología y la diferencia está en el contexto de uso. En física

la MPS se introdujo para representar estados cuánticos de muchos cuerpos, mientras que en análisis numérico y aprendizaje automático el formato TT se emplea como herramienta de compresión y parametrización eficiente de tensores de alta dimensión [21].

Los índices internos de la cadena reciben el nombre de rangos TT o dimensiones de enlace, y determinan cuánta información puede transmitirse entre las distintas partes de la cadena. Dimensiones pequeñas dan un modelo muy compacto pero con capacidad limitada para representar correlaciones entre posiciones alejadas; al aumentarlas, la representación gana expresividad a costa de más parámetros y contracciones más costosas. En aplicaciones de aprendizaje automático, la dimensión de enlace actúa por tanto como un hiperparámetro que regula la complejidad del modelo [7].

Una ventaja relevante del formato TT es que puede construirse aplicando descomposiciones en valores singulares de forma sucesiva a distintas matricizaciones del tensor. Este procedimiento, conocido como TT-SVD, proporciona una forma numéricamente estable de obtener una aproximación con rangos controlados [21], y conecta la MPS con la SVD introducida anteriormente: no es solo una arquitectura inspirada en la física, sino también una herramienta de reducción de dimensionalidad.

Desde el punto de vista físico, la MPS es la representación más natural de los estados fundamentales de los sistemas cuánticos unidimensionales con interacciones locales, lo que explica su relevancia en este campo. Una propiedad característica es que las correlaciones que puede representar decaen exponencialmente con la distancia entre posiciones: la función de correlación entre las posiciones i y j satisface:

$$\langle O_i O_j \rangle - \langle O_i \rangle \langle O_j \rangle \sim e^{-|i-j|/\xi} \quad (3.15)$$

donde ξ es una longitud de correlación que depende de la dimensión de enlace D [7]. Por ello la MPS captura con eficiencia las correlaciones de corto alcance, pero puede requerir dimensiones de enlace elevadas para las de largo alcance.

No obstante, la MPS organiza la información en una cadena. Esto es natural para datos secuenciales, pero menos adecuado para imágenes, que poseen una estructura espacial bidimensional. Aplicar una MPS a una imagen obliga a ordenar sus posiciones de entrada en una secuencia. Esa ordenación puede conservar parcialmente la vecindad espacial, por ejemplo mediante un recorrido en zigzag, pero transforma inevitablemente una geometría bidimensional en una unidimensional. Esta limitación es una de las motivaciones para considerar topologías jerárquicas, como las Tree Tensor Networks, en problemas de visión artificial.

Una forma especialmente interesante de utilizar una MPS en aprendizaje supervisado consiste en emplearla como parametrización del tensor de pesos de un clasificador. En un modelo lineal convencional, la entrada se representa mediante un vector de características que se multiplica por una matriz de pesos. En el enfoque tensorial, la entrada se transforma primero en un tensor de alto orden mediante un mapa local de características y el tensor de

pesos se representa de forma comprimida mediante una MPS [7]. De manera esquemática, si una entrada x se descompone en varias posiciones $x_1, \dots, x_N$, a cada una se le aplica un mapa local

$$\phi(x_j) \in \mathbb{R}^d,$$

con d la dimensión local, y la representación completa de la entrada se obtiene como producto tensorial de todos los vectores locales:

$$\Phi(x) = \phi(x_1) \otimes \phi(x_2) \otimes \dots \otimes \phi(x_N).$$

Aunque formalmente $\Phi(x)$ pertenece a un espacio de dimensión d^N , no es necesario construirlo de forma explícita, porque la estructura de la red permite trabajar con esta representación de manera implícita mediante contracciones sucesivas. Para clasificación multiclase se introduce un índice adicional asociado a la etiqueta de clase, de modo que la MPS representa un tensor de pesos W^ℓ y, al contraerlo con la representación de entrada, se obtiene una puntuación por clase:

$$f^\ell(x) = W^\ell \cdot \Phi(x).$$

La clase predicha es la de mayor puntuación. En el lenguaje habitual del *deep learning*, estas puntuaciones se interpretan como los valores previos a la normalización probabilística final del clasificador. Así, la MPS no se limita a comprimir un tensor ya conocido, sino que define directamente la forma de los parámetros que se optimizan, y su dimensión de enlace controla la complejidad del clasificador.

Este esquema fue explorado por Stoudenmire y Schwab, que aplicaron una MPS a la clasificación de dígitos manuscritos sobre MNIST, ordenando los píxeles en una secuencia mediante un recorrido en zigzag para preservar en parte la proximidad espacial, y alcanzaron un error de test inferior al 1 % [7]. Este resultado ilustra el potencial de la MPS como clasificador entrenable, pero también se hace evidente la necesidad de linealizar una imagen bidimensional antes de introducirla en la red, que es su gran limitación.

3.6.2 Tree Tensor Network

Una *Tree Tensor Network* (TTN) organiza los tensores siguiendo una estructura jerárquica en forma de árbol. En lugar de propagar la información a lo largo de una cadena, asocia los tensores de la capa más baja (las *hojas* o *leaves* en inglés) a las posiciones de entrada y los contrae por grupos hacia tensores de capas superiores, reduciendo progresivamente el número de índices hasta llegar a un único tensor raíz que produce la salida. Los tensores de los niveles inferiores combinan información local, mientras que los superiores integran representaciones cada vez más globales, de modo que la red construye una jerarquía de

características sobre la que se realiza la clasificación.

La diferencia esencial con una MPS es la forma en que se propagan las dependencias entre posiciones de entrada. En una cadena, dos posiciones alejadas solo se comunican a través de todos los tensores intermedios; en un árbol, en cambio, basta ascender unos pocos niveles hasta un nodo común, de manera que la distancia efectiva entre dos hojas crece como $\mathcal{O}(\log N)$ en lugar de linealmente [5, 10]. Esta geometría hace que la TTN sea más eficiente que la MPS para combinar información lejana y, por tanto, para capturar correlaciones de largo alcance.

Al igual que en una MPS, la dimensión de los índices internos regula el compromiso entre compacidad y expresividad. Si es demasiado pequeña, la red pierde información relevante durante la compresión jerárquica; si es demasiado grande, gana capacidad pero aumentan el número de parámetros y el coste de entrenamiento. Existe, además, una decisión estructural que no es neutra: la forma concreta del árbol fija de antemano qué grupos de entradas se combinan primero y cuáles solo se encuentran en niveles superiores, lo que condiciona el tipo de dependencias que el modelo representa con mayor facilidad. En problemas de imágenes esta cuestión es especialmente relevante, ya que la organización espacial de la entrada influye en qué regiones conviene agrupar antes.

En física, la TTN se emplea para representar estados con una estructura de entrelazamiento jerárquica, como la que aparece en ciertos sistemas con simetría de escala. En *machine learning*, su organización en árbol resulta natural para datos con estructura jerárquica intrínseca [5]. Esta correspondencia es la que motiva su uso en este trabajo. Aunque una TTN no reproduce de forma exacta la geometría bidimensional de una imagen, sí ofrece un mecanismo para agrupar posiciones de entrada y combinar la información de manera progresiva. Por ello la TTN no se aplica aquí directamente sobre los píxeles, sino sobre una secuencia de vectores de características obtenida tras una etapa inicial de extracción visual, un planteamiento híbrido que se detalla en la metodología.

La estructura de árbol también introduce una limitación. Al no contener ciclos, la información entre dos ramas distintas solo puede propagarse a través de su nodo ancestro común, lo que restringe la capacidad de la red para capturar correlaciones entre nodos situados en ramas separadas. Esta restricción es la que motivó el desarrollo de arquitecturas más sofisticadas como la MERA.

3.6.3 MERA

La *Multi-scale Entanglement Renormalization Ansatz* (MERA) fue introducida por Vidal en 2007 como una Tensor Network capaz de representar eficientemente los estados fundamentales de sistemas cuánticos en los que las correlaciones decaen como una ley de potencias en lugar de exponencialmente [22]. Puede entenderse como una extensión de las redes jerárquicas tipo árbol: al igual que una TTN, organiza la información en niveles sucesivos, pero introduce un elemento adicional característico, los *disentangles* (desentrelazadores).

En cada capa de la estructura jerárquica, antes de reducir el número de grados de libertad, los disentanglers actúan sobre pares de posiciones vecinas para eliminar el entrelazamiento de corto alcance. A continuación, los tensores de renormalización (isometrías) agrupan varios grados de libertad en una representación más compacta y capturan las correlaciones a escalas mayores. La alternancia de capas de disentanglers y de renormalización es lo que permite a la MERA representar correlaciones a todas las escalas de longitud simultáneamente [22, 5].

Esta organización está estrechamente relacionada con el grupo de renormalización. Cada capa puede interpretarse como una descripción efectiva del sistema a una escala de longitud distinta, de las más locales en los niveles inferiores a las más globales en los superiores. En esta lectura, mientras una MPS reproduce esencialmente la geometría unidimensional de la cadena, la MERA introduce una geometría adicional asociada a la escala, que conecta posiciones alejadas a través de sus capas superiores [10]. Por ello, donde una MPS podría requerir una dimensión de enlace que crece con el tamaño del sistema, la estructura multiescala de la MERA captura estas correlaciones de forma más natural.

En *machine learning*, esta capacidad para tratar dependencias a múltiples escalas resulta sugerente para datos como las imágenes, que contienen patrones organizados a distintos niveles. Sin embargo, la presencia de disentanglers y el mayor coste de sus contracciones hacen que la MERA sea bastante más compleja de implementar y entrenar que una MPS o una TTN. Por esta razón, en este trabajo se considera principalmente como una topología relevante desde el punto de vista teórico, mientras que la implementación experimental se apoya en una arquitectura jerárquica basada en TTN, situada así dentro de una familia más amplia de redes tensoriales jerárquicas.

3.6.4 PEPS

Los *Projected Entangled Pair States* (PEPS) son la generalización más natural del MPS a dos dimensiones. Mientras que en la topología MPS cada tensor se conecta a lo sumo con dos vecinos, en PEPS cada tensor se conecta con cuatro vecinos, formando una red bidimensional que refleja directamente la geometría de una rejilla cuadrada [23].

Para un sistema bidimensional de $M \times N$ sitios, cada tensor local en PEPS tiene un índice físico de dimensión d y cuatro índices virtuales (arriba, abajo, izquierda, derecha) de dimensión D . Esto permite capturar de forma natural las correlaciones espaciales bidimensionales que caracterizan, por ejemplo, los estados fundamentales de sistemas cuánticos definidos sobre rejillas planas [5].

Sin embargo, esta ventaja geométrica tiene un coste. La contracción exacta de una red PEPS es un problema computacionalmente intratable, concretamente #P-difícil [9]; a diferencia del MPS, donde la contracción exacta tiene un coste polinomial. En la práctica, esto obliga a utilizar métodos de contracción aproximada, lo que introduce errores adicionales y limita el tamaño de las dimensiones de enlace que pueden utilizarse [5]. Existen aplicaciones de PEPS a clasificación supervisada en conjuntos como MNIST y

Fashion-MNIST [8].

Topología	Parámetros típicos	Ventaja principal	Limitación práctica
MPS / TT	$\mathcal{O}(NdD^2)$	Contracción eficiente y literatura consolidada.	Geometría 1D para datos que pueden ser 2D.
TTN	$\mathcal{O}(ND^3)$	Agregación jerárquica y distancia logarítmica.	Interacciones limitadas entre ramas separadas.
PEPS	$\mathcal{O}(NdD^4)$	Geometría 2D natural.	Contracción exacta intratable en general.
MERA	$\mathcal{O}(ND^4)$	Correlaciones multiescala.	Mayor complejidad estructural y computacional.

Tabla 3.2: Comparativa sintética de topologías relevantes de Tensor Networks. Los conteos de parámetros se expresan a orden de magnitud; en las filas de TTN y MERA se omite por simplicidad el factor asociado a la dimensión local d , que sí se indica explícitamente en MPS/TT y PEPS.

3.7 Tensor Networks en aprendizaje automático

Aunque las Tensor Networks surgieron principalmente en el contexto de la física cuántica y del análisis numérico, en los últimos años han adquirido interés dentro del *machine learning*. La razón principal es que muchos problemas de *machine learning* implican trabajar con objetos de alta dimensión: imágenes, secuencias, representaciones intermedias de redes neuronales o tensores de pesos con muchos parámetros. En estos casos, las redes tensoriales ofrecen una forma estructurada de representar interacciones entre muchas variables sin tener que construir explícitamente todos los coeficientes del tensor completo.

De forma general, las aplicaciones de Tensor Networks en aprendizaje automático pueden dividirse en dos enfoques. El primero consiste en utilizarlas como herramientas de compresión. En este caso, una capa densa, una convolución o una representación de datos se sustituye por una descomposición tensorial de menor número de parámetros. Así, el modelo mantiene una estructura similar a la arquitectura original, pero algunos de sus tensores de pesos se reemplazan por formatos como CP, Tucker, Tensor Train o Tensor Ring. Este enfoque busca reducir memoria y número de parámetros, aunque no siempre implica una aceleración proporcional, ya que las contracciones tensoriales también tienen un coste computacional [5].

El segundo enfoque consiste en utilizar la Tensor Network como arquitectura entrenable en sí misma. En lugar de comprimir una red ya existente, se define un modelo cuya estructura principal es una red tensorial, y son sus propios tensores los que se ajustan a partir de los datos. El ejemplo canónico es el clasificador basado en MPS descrito en la subsección

anterior, en el que la entrada se transforma en una representación tensorial mediante un mapa de características que luego se contrae con la red. Es este segundo enfoque, y no el de compresión, el que se adopta en este trabajo.

La aplicación a visión artificial añade una dificultad propia. Las imágenes tienen una estructura espacial bidimensional, mientras que topologías como la MPS son intrínsecamente unidimensionales y las jerárquicas como la TTN tampoco reproducen de forma exacta esa geometría. Linealizar la imagen en una secuencia, aunque sea con un recorrido en zigzag, pierde parte de esa estructura. Por ello, la estrategia adoptada en este trabajo es crear una etapa inicial de extracción visual produciendo una secuencia de vectores de características y, sobre ella, la red tensorial jerárquica combina la información de forma estructurada. De este modo, la TTN no tiene que aprender desde cero los patrones locales básicos de la imagen, algo especialmente relevante en imágenes reales, donde el número de valores de entrada es mucho mayor que en conjuntos clásicos como MNIST. El detalle de esta arquitectura híbrida se desarrolla en el capítulo de metodología.

Desde esta perspectiva, las Tensor Networks no deben entenderse como sustitutos automáticos de las redes convolucionales o de otras arquitecturas estándar de visión artificial. Su interés reside más bien en ofrecer una forma alternativa de imponer estructura sobre el modelo, controlar el número de parámetros y estudiar cómo se combinan las representaciones internas. Por ello, en este TFG la comparación con modelos convencionales, como ResNet50, se plantea como una referencia para contextualizar el rendimiento del modelo tensorial, no como una demostración de superioridad general.

En resumen, las Tensor Networks proporcionan un puente natural entre herramientas matemáticas propias de la física y técnicas actuales de *deep learning*. Su uso en clasificación de imágenes permite explorar arquitecturas con una estructura interna distinta a la de los modelos convencionales, pero también exige tomar decisiones cuidadosas sobre la representación de entrada, la topología de la red, la dimensión de enlace, la regularización y el procedimiento de entrenamiento.

Capítulo 4. METODOLOGÍA

En los capítulos anteriores se han introducido los conceptos matemáticos necesarios para entender el *machine learning*, el *deep learning* y las redes tensoriales. En este capítulo se utilizan esos conocimientos para explicar cómo se construye un modelo de clasificación de imágenes basado en redes tensoriales.

El capítulo describe, de forma reproducible, todas las etapas del trabajo: la estructura general del *pipeline* y la organización del proyecto, la preparación del conjunto de datos ChemEq25, el preprocesamiento y el aumento de datos, la transformación de la imagen en una secuencia (*embedding*), la arquitectura tensorial propuesta, el procedimiento de entrenamiento y regularización, el protocolo de comparación con la arquitectura de referencia ResNet50, las métricas empleadas y las condiciones de reproducibilidad y ejecución en servidor.

4.1 Visión general del pipeline

Para construir un modelo de clasificación de imágenes en PyTorch se sigue una estructura común sobre la que se apoyan todos los modelos. Un conjunto de datos devuelve una imagen y su etiqueta; unas transformaciones preparan la imagen; un agrupador de muestras (*DataLoader*) las reúne en lotes (*batches*) para procesarlas de forma eficiente; el modelo produce, para cada imagen, una puntuación por clase (los *logits*); una función de pérdida compara esas puntuaciones con la etiqueta correcta; y, finalmente, el optimizador actualiza los parámetros del modelo en la dirección que reduce el error, una operación que se conoce como retropropagación (*backpropagation*).

`datos ->transformaciones ->DataLoader ->modelo ->pérdida ->optimizador`

Esta estructura es importante porque separa responsabilidades. El modelo tensorial no necesita saber de dónde procede una imagen, ni el conjunto de datos necesita conocer cómo opera internamente la red tensorial. Cada parte tiene una tarea clara. El conjunto de datos entrega imágenes y etiquetas, el modelo recibe un lote de imágenes y devuelve una matriz de puntuaciones (*logits*), y el bucle de entrenamiento se encarga de calcular la pérdida y actualizar los pesos. Esta separación es la que permite, por ejemplo, cambiar la forma de representar la imagen o comparar dos arquitecturas distintas manteniendo el mismo flujo de datos.

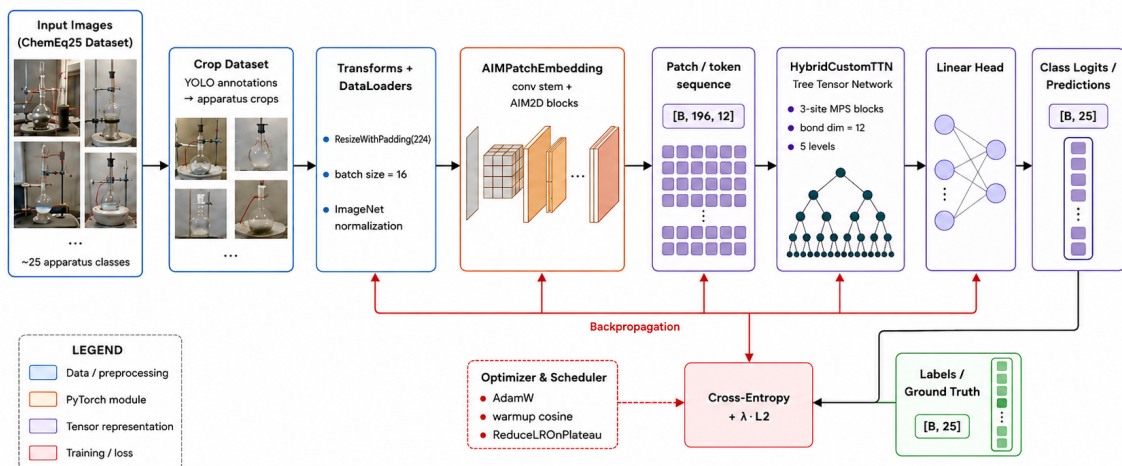


Figura 4.1: Esquema general del pipeline desarrollado en el trabajo. A partir de las imágenes originales y sus anotaciones se generan recortes individuales de objetos, se aplican transformaciones y se agrupan en lotes. Cada imagen procesada entra en la etapa de *embedding*, que produce una secuencia compatible con la red tensorial. La salida de la red tensorial pasa por una cabeza de clasificación y se compara con la etiqueta real mediante la función de pérdida. Las flechas inferiores representan la actualización de parámetros por retropropagación.

El código del proyecto se organizó de forma modular, separando responsabilidades en distintos módulos diferenciados. Esta organización hace que el proyecto sea mucho más manejable y permite modificar una parte sin alterar el resto. El detalle de la estructura de carpetas se recoge en los anexos.

Por último, cada configuración de los experimentos queda definida por un archivo de configuración en formato YAML (*Yet Another Markup Language*), que fija todos los parámetros relevantes, como el conjunto de datos, el tamaño de imagen, el tipo de *embedding*, la dimensión local y la dimensión de enlace, el número de épocas, el ritmo de aprendizaje, la regularización, el *scheduler* y la semilla aleatoria, entre otros. De este modo, un experimento puede repetirse o compararse leyendo simplemente su configuración, sin depender de notas externas. Esta forma de trabajar es la base de la trazabilidad del proyecto.

4.2 Preparación del dataset ChemEq25

El dataset utilizado en este proyecto es ChemEq25, también llamado en el código *Chemistry Lab Apparatus* [13]. Este dataset contiene imágenes de material de laboratorio y anotaciones asociadas a los objetos presentes en cada imagen.

El problema no se trató como una clasificación directa de imágenes completas debido a que, en muchas imágenes aparece más de un objeto, y esos objetos pueden pertenecer a clases distintas. Si se introdujera la imagen completa en el clasificador, la etiqueta sería ambigua. Por eso se reformuló el problema como clasificación por instancia, en el que cada anotación YOLO (*You Only Look Once*) genera un recorte o *crop* de un único objeto, y ese *crop* se asocia a una clase.

Tras convertir las anotaciones en muestras individuales, los tamaños de los splits fueron:

Split	Número de crops
Entrenamiento	4863
Validación	1393
Test	704

El procedimiento de lectura e interpretación (*parser* en inglés) de las anotaciones tuvo que contemplar dos casos. Algunas etiquetas seguían el formato YOLO habitual:

```
class_id x_center y_center width height
```

pero otras contenían más coordenadas, correspondientes a polígonos de segmentación. Para que todo pudiera tratarse de manera uniforme, las anotaciones de cinco coordenadas se interpretaron como cajas YOLO y las anotaciones con más coordenadas se transformaron en la caja rectangular envolvente del polígono.

Después de obtener el *crop*, todas las imágenes deben convertirse a un tamaño común para entrar al modelo. Inicialmente se podía usar un `Resize(224, 224)` directo, pero esto deforma objetos alargados como pipetas, tubos de ensayo o varillas. Por esa razón se implementó una opción para preservar la relación de aspecto y rellenar hasta obtener una imagen cuadrada. El color de relleno se escogió cercano a la media de *ImageNet*, de forma que tras la normalización el relleno (*padding*) no quedara como un borde artificial demasiado dominante.

4.3 Transformaciones y aumento de datos

El preprocesamiento de las imágenes se separó entre entrenamiento y evaluación. La parte de evaluación consta de un proceso de validación y otro de test. En esta parte no se aplican transformaciones aleatorias, porque se quiere medir el comportamiento del modelo sobre datos estables. En el entrenamiento sí se usa *data augmentation*, con el objetivo de que el modelo no memorice una única apariencia de cada objeto.

Este proceso de aumento de datos se hizo configurable, es decir, que el usuario puede decidir si quiere aplicarlo o no. En ChemEq se usaron opciones como:

- `augmentation_name = none`, sin aumentos aleatorios.

- `augmentation_name = basic`, con transformaciones suaves ya existentes.
- `augmentation_name = light`, añadiendo rotaciones pequeñas y `RandomErasing` bajo.

La configuración final más relevante usó la configuración `light`, rotaciones de 7 grados y `RandomErasing` con probabilidad 0,10. La idea fue introducir variabilidad sin destruir la geometría del objeto, porque en material de laboratorio la forma es precisamente una parte importante de la clase.

4.4 Representación de la imagen como secuencia

Una red tensorial no procesa directamente una imagen con forma $[C, H, W]$ (canales, alto y ancho). Su entrada natural es una secuencia ordenada de vectores locales. Dicho de otra forma, son un conjunto de posiciones de entrada en el que cada posición aporta un vector de dimensión fija. Por eso, antes de la red tensorial es necesaria una etapa que transforme la imagen en esa secuencia:

$$[B, C, H, W] \rightarrow [B, L, \text{local_dim}]$$

donde B es el número de imágenes que se procesan a la vez (un *batch* o lote), L es el número de posiciones de entrada de la red tensorial y `local_dim` es la dimensión del vector asociado a cada posición. Esta etapa se denomina *embedding* de la imagen y es una parte muy importante del modelo, ya que condiciona su rendimiento de manera considerable, porque determina qué información espacial llega a la red tensorial y en qué forma se le entrega.

El primer enfoque que se probó dividía la imagen en cuadrados y proyectaba cada uno de forma independiente con una transformación lineal. Este enfoque resultó insuficiente, ya que al construir la secuencia se pierde la estructura bidimensional de la imagen y la relación de vecindad entre cuadrados contiguos. Por ese motivo, la versión final del proyecto utiliza un *embedding* que conserva la estructura espacial hasta el último momento, denominado en el código `AIMPatchEmbedding`. A continuación se describe su funcionamiento.

El `AIMPatchEmbedding` no trocea la imagen, sino que la procesa como un mapa bidimensional que se va reduciendo de tamaño progresivamente. Solo en el último paso ese mapa se aplanado para formar la secuencia que recibe la red tensorial. El flujo completo de dimensiones, para imágenes de entrada de 224×224 , es:

```
[B, 3, 224, 224]
->stem convolucional ->[B, C_aim, 14, 14]
    ->bloques AIM ->[B, C_aim, 14, 14]
->convolución 1x1 ->[B, local_dim, 14, 14]
->aplanado + zigzag ->[B, 196, local_dim]
```

La imagen pasa primero por un bloque convolucional inicial, denominado habitualmente *stem*, cuya función es extraer características locales básicas y reducir la resolución espacial de la entrada. Después por una serie de bloques AIM que mantienen el mapa espacial e introducen interacciones entre canales y posiciones, y finalmente por una proyección que ajusta el número de canales a la dimensión local que espera la red tensorial. El mapa resultante de 14×14 se aplanan en una secuencia de 196 posiciones de entrada.

El *stem* es la etapa inicial que sustituye el corte en cuadrados por una pequeña red convolucional. Una convolución es una operación que desliza un filtro pequeño sobre la imagen para detectar patrones locales (bordes, texturas, formas) reutilizando los mismos pesos en todas las posiciones. El *stem* se construye como una pila de bloques, cada uno formado por una convolución con paso (*stride*) dos, seguida de una normalización por lotes (*Batch-Norm*, que reescala las activaciones usando estadísticas del propio *batch* para estabilizar el entrenamiento) y una función de activación ReLU (que anula los valores negativos e introduce no linealidad).

El paso dos significa que el filtro se desplaza de dos en dos píxeles, de modo que cada bloque reduce a la mitad el alto y el ancho del mapa. El número de bloques viene dado por el hiperparámetro `patch_size`, que en este *embedding* ya no representa el tamaño de un cuadrado, sino el factor total de reducción espacial. Debe ser una potencia de dos, y el número de bloques es $\log_2(\text{patch_size})$. Con `patch_size = 16` se encadenan cuatro bloques, que reducen la imagen de la siguiente forma:

224 ->112 ->56 ->28 ->14

El primer bloque transforma los tres canales de color en `aim_channels` canales, y los siguientes mantienen ese número. El resultado es un mapa `[B, aim_channels, 14, 14]` en el que cada una de las 14×14 posiciones del mapa pasará a ser una posición de entrada de la red tensorial.

Sobre el mapa producido por el *stem* se aplican uno o varios bloques AIM, inspirados en el trabajo *Deep Tree Tensor Networks for Image Recognition* [11]. Cada bloque recibe un mapa `[B, C, H, W]` y devuelve otro de la misma forma, lo que permite encadenar varios y, como se verá, añadir una conexión residual.

La idea central del bloque es construir dos ramas que aplican las mismas dos familias de operaciones, pero en orden inverso:

- **Rama 1:** primero una transformación lineal que mezcla los canales en cada píxel y una convolución que aplica un filtro espacial independiente a cada canal sin mezclar información entre canales.
- **Rama 2:** primero una convolución agrupada (*grouped*), que procesa cada canal de entrada por separado para generar varios canales nuevos, y después una transformación lineal que mezcla los canales.

Internamente ambas ramas expanden el número de canales por un factor configurable (`expansion_ratio`), lo que da al bloque más capacidad de representación antes de volver a comprimir. Las salidas de las dos ramas se combinan mediante un producto de Hadamard, esto es, una multiplicación elemento a elemento de dos tensores de la misma forma:

$$\text{interaction} = f1(x) * f2(x)$$

Este producto es la clave del bloque y el motivo por el que encaja bien delante de una red tensorial. Las dos ramas se construyen sin término independiente (*bias*) y sin funciones de activación intermedias, de modo que cada una es una transformación lineal de la entrada. Al multiplicarlas elemento a elemento, el resultado es bilineal, es decir, cuadrático en la entrada. Se introduce así una interacción multiplicativa entre las características de la imagen, justo el tipo de operación que las redes tensoriales expresan de forma natural, y sin recurrir a no linealidades del tipo ReLU o sigmoide. El nombre del módulo en la literatura, *Antisymmetric Interaction Module* (AIM), alude precisamente a esa construcción en orden inverso de las dos ramas antes del producto.

Tras el producto, el resultado se normaliza con una normalización por capas (*LayerNorm*, que reescala cada vector de canales para estabilizar su escala y distribución) y se proyecta de vuelta al número original de canales mediante una convolución 1×1 , de manera que la salida pueda sumarse con la entrada del bloque. Esta suma es una conexión residual; en lugar de reemplazar la entrada, el bloque aprende una corrección que se le añade,

$$\text{salida} = x + s * \text{correccion}$$

donde s es un factor de escala entrenable inicializado a un valor pequeño. Inicializarlo pequeño hace que, al comienzo del entrenamiento, el bloque se comporte casi como la identidad y no desestabilice las activaciones que recibirá la red tensorial. La etapa AIM aplica `num_aim_blocks` bloques de este tipo de forma consecutiva.

Después de los bloques AIM, una última convolución 1×1 ajusta el número de canales desde `aim_channels` hasta `local_dim`, la dimensión que espera la red tensorial en cada posición. El mapa resultante, de forma `[B, local_dim, 14, 14]`, se reordena para situar los canales al final y se aplana convirtiendo la rejilla espacial en una secuencia:

$$[B, \text{local_dim}, 14, 14] \rightarrow [B, 196, \text{local_dim}]$$

El orden en que se recorren las 196 posiciones no es indiferente. Un recorrido por filas (*row-major*) saltaría del final de una fila al principio de la siguiente, introduciendo una discontinuidad espacial entre posiciones consecutivas de la secuencia. Para suavizar este efecto se emplea un recorrido en zigzag. Las filas pares se recorren de izquierda a derecha y las impares de derecha a izquierda, de modo que dos posiciones contiguas en la secuencia tienden a ser también vecinas en la imagen. Cada una de las 196 posiciones resultantes es

un vector local de dimensión `local_dim` y constituye una posición de entrada de la red tensorial descrita en la siguiente sección.

En conjunto, este *embedding* conserva la estructura espacial bidimensional de la imagen hasta el momento mismo del aplanado, en lugar de destruirla al inicio. Las convoluciones, al compartir pesos entre posiciones, actúan como un sesgo inductivo y como una forma de regularización, y la interacción multiplicativa del bloque AIM aporta una no linealidad coherente con el lenguaje de las redes tensoriales. La conexión residual, por su parte, permite que la etapa inicial parta de un comportamiento próximo a la identidad y establezca el entrenamiento del modelo completo.

4.5 Arquitectura tensorial propuesta

La secuencia de 196 vectores locales producida por el *embedding* es la entrada de la red tensorial entrenable, que procesa la información hasta obtener un único vector con el que se decide la clase. El tipo de red tensorial es una jerárquica de tipo *Tree Tensor Network* (TTN), denominada en el código `HybridCustomTTN`, que se describe a continuación.

La idea de una TTN es organizar el cálculo como un árbol. Las posiciones de entrada se agrupan de tres en tres, cada grupo se contrae con un pequeño bloque tensorial que produce un solo vector de salida, y ese vector pasa a ser una entrada del siguiente nivel del árbol. El proceso se repite, reduciendo el número de posiciones en cada nivel, hasta llegar a un único vector raíz que resume toda la imagen. Que los grupos sean de tres elementos es lo que se denomina aridad tres del árbol.

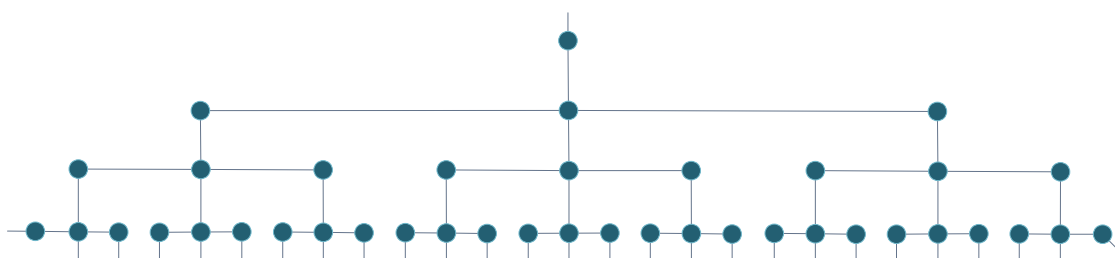


Figura 4.2: Diagrama de la TTN. Es una imagen orientativa, ya que representa tres niveles cuando la arquitectura empleada tiene cinco. Las posiciones de entrada se agrupan de tres en tres y se contraen jerárquicamente hasta un único vector raíz. Imagen de elaboración propia con la librería `Tensor-Network-Editor`, creada por Alejandro Mata Ali. [3]

En el lenguaje de redes tensoriales conviene distinguir dos tipos de índices. Los índices físicos (o de entrada) son los que conectan la red con los datos; por cada posición de entrada hay un índice físico cuya dimensión es la del vector local que llega a esa posición. En el primer nivel esa dimensión es `local_dim`, la que produce el *embedding*. Los índices virtuales (o de enlace) son internos; conectan unos tensores con otros dentro de la red y no tocan los datos. Su dimensión común se denomina dimensión de enlace (`bond_dim`).

La dimensión de enlace es el principal parámetro que controla la capacidad del modelo. Cuanto mayor es `bond_dim`, más información puede transmitirse a través de cada conexión interna y más ricas son las correlaciones que la red puede representar entre distintas partes de la imagen, a costa de más parámetros y más coste de cálculo. Junto con la topología del árbol, es la decisión de diseño que regula el equilibrio entre capacidad y eficiencia.

La unidad básica de la TTN es un bloque tensorial de tres nodos, con la estructura de una MPS corta. Cada bloque está formado por un nodo izquierdo, uno central y uno derecho. El nodo izquierdo tiene un índice físico de entrada y un índice virtual que lo une al central. El nodo central tiene un índice físico de entrada, dos índices virtuales (uno hacia el nodo izquierdo y otro hacia el derecho) y un índice de salida. El nodo derecho tiene un índice físico de entrada y un índice virtual que lo une al central.

El índice de salida se sitúa en el nodo central y tiene dimensión `bond_dim`; es el que conecta el bloque con el nivel superior del árbol. Cada bloque recibe tres vectores de entrada (uno por nodo) y los contrae junto con sus tres tensores para producir un único vector de salida. Si se denota por $v^{(1)}, v^{(2)}, v^{(3)}$ a los tres vectores de entrada, por A, C y B a los tensores de los nodos izquierdo, central y derecho, y se usan las letras a, b para los índices virtuales, i, j, k para los físicos y β para el de salida, la salida del bloque es:

$$o_{\beta} = \sum_{a,b} \sum_{i,j,k} A_{ai} C_{abj\beta} B_{bk} v_i^{(1)} v_j^{(2)} v_k^{(3)}. \quad (4.1)$$

Los índices virtuales a y b tienen dimensión `bond_dim`, los físicos i, j, k tienen la dimensión de entrada del nivel y el índice de salida β tiene de nuevo dimensión `bond_dim`. La contracción combina de forma multiplicativa la información de los tres vectores de entrada, lo que conecta directamente con la motivación de usar redes tensoriales. La salida depende de productos entre las componentes de la entrada, no solo de sumas ponderadas.

Los tres tensores de cada bloque son parámetros entrenables del modelo. En la implementación esto exigió declararlos de un tipo específico de TensorKrowch (`ParamNode`), de modo que sus valores se registran como parámetros del modelo y reciben gradientes durante el entrenamiento; si se hubieran creado como nodos ordinarios, no aparecerían como parámetros entrenables. Sus valores se inicializan con ruido aleatorio de desviación típica pequeña (0,02), un valor que mantiene las contracciones numéricamente estables a medida que se asciende por los niveles del árbol.

El número de posiciones de un nivel no siempre es múltiplo de tres. Cuando el último grupo queda incompleto, las posiciones que faltan se rellenan con nodos constantes, no entrenables, cuyo vector es $[1, 0, \dots, 0]$. Estos nodos de relleno (*padding*) completan el grupo para que el bloque pueda contraerse, pero no aportan parámetros ni introducen información, ya que su vector es fijo y neutro. De esta forma la estructura jerárquica se mantiene regular sin distorsionar el aprendizaje.

Cada nivel del árbol se implementó como una red tensorial independiente (`TTNLevel`) que contrae únicamente su propio nivel de bloques de tres nodos y devuelve un tensor de Py-

Torch ordinario. El enlace entre un nivel y el siguiente no se hace dentro de TensorKrowch, sino en PyTorch: la salida de un nivel, antes de entrar en el siguiente, pasa por una pequeña transformación estándar formada por una normalización por capas (*LayerNorm*), una capa lineal y una activación ReLU:

nivel ->LayerNorm ->Linear ->ReLU ->nivel siguiente

Esta combinación es la razón del nombre *HybridCustomTTN*. Es híbrida porque las contracciones tensoriales se realizan con TensorKrowch, mientras que las transformaciones intermedias, el entrenamiento y la integración del modelo se gestionan con PyTorch. Las transformaciones entre niveles aportan flexibilidad adicional y ayudan a estabilizar las escalas de los vectores al pasar de un nivel a otro.

Partiendo de las 196 posiciones de entrada y agrupando de tres en tres, el número de posiciones se reduce nivel a nivel de la forma

196 ->66 ->22 ->8 ->3 ->1,

de modo que la arquitectura empleada consta de cinco niveles tensoriales y cuatro transformaciones intermedias. La salida del último nivel es un único vector raíz de dimensión `bond_dim` que condensa la información de toda la imagen.

El vector raíz no es todavía una predicción. Para obtener una puntuación por clase se aplica una cabeza de clasificación, que en su versión más sencilla es una única capa lineal que transforma el vector raíz de dimensión `bond_dim` en un vector de dimensión igual al número de clases (25 en ChemEq25). Las componentes de ese vector son las puntuaciones sin normalizar de cada clase (*logits*), que en la etapa de entrenamiento se compararán con la etiqueta real mediante la función de pérdida descrita más adelante. Opcionalmente, esta cabeza puede sustituirse por un pequeño perceptrón multicapa con una capa oculta, una activación y regularización por *dropout*, aunque la versión lineal es la que define el modelo de referencia.

4.6 Entrenamiento y regularización

Una vez definidos el *embedding* y la red tensorial, el modelo se entrena ajustando sus parámetros para que clasifique correctamente los recortes. En cada paso, el modelo procesa un *batch* de imágenes y produce sus puntuaciones por clase; una función de pérdida mide cuánto se equivoca, y a partir de ese error se calculan los gradientes que indican cómo modificar cada parámetro para reducirlo. Esta sección explica las decisiones tomadas en ese proceso: la función de pérdida, las técnicas de regularización y el control del ritmo de aprendizaje.

Para entrenar el clasificador se empleó la entropía cruzada, una función de pérdida habitual en problemas de clasificación con varias clases. El modelo no devuelve directamente probabilidades, sino una puntuación para cada clase (los *logits* introducidos antes). Durante

el cálculo de la pérdida, esas puntuaciones se transforman internamente en probabilidades normalizadas y se penaliza al modelo cuando asigna una probabilidad baja a la clase correcta. Cuanto más confiado está el modelo en la clase verdadera, menor es la pérdida.

En la implementación, esta función (`CrossEntropyLoss` en PyTorch) combina internamente la normalización en probabilidades (*softmax*) y la comparación con la etiqueta correcta. Por eso el modelo no necesita aplicar esa normalización en su capa de salida, basta con que produzca los *logits*. Esta combinación interna evita duplicar operaciones y hace el entrenamiento numéricamente más estable.

Durante el entrenamiento se aplicó además un suavizado de etiquetas (*label smoothing*). En lugar de exigir al modelo que asigne toda la probabilidad a la clase correcta y exactamente cero al resto, el objetivo se relaja ligeramente, dejando una pequeña parte de probabilidad repartida entre las demás clases. Esto evita que el modelo se vuelva excesivamente confiado y suele mejorar su capacidad de generalizar a imágenes nuevas.

El suavizado solo se usa en entrenamiento. En la evaluación se emplea la entropía cruzada normal, sin suavizado, de modo que la pérdida de validación (`eval_loss`) sigue midiendo el ajuste real del modelo frente a las etiquetas verdaderas y permite comparar experimentos de forma coherente.

El optimizador es el algoritmo que decide cómo actualizar los parámetros del modelo a partir de los gradientes. Se utilizó AdamW (`AdamW`), una variante muy extendida del optimizador Adam. Adam adapta automáticamente el tamaño del paso de actualización para cada parámetro, usando estimaciones del comportamiento reciente de sus gradientes, lo que suele acelerar y estabilizar el entrenamiento frente a un descenso de gradiente simple. La variante AdamW incorpora además, de forma adecuada, una penalización que tiende a mantener pequeños los pesos del modelo (*weight decay*), descrita a continuación.

La regularización agrupa las técnicas que evitan que el modelo se limite a memorizar las imágenes de entrenamiento (sobreajuste) en lugar de aprender características que generalicen. En este trabajo se combinaron dos mecanismos. El primero es la penalización por reducción de pesos (*weight decay*) que incorpora AdamW, que empuja suavemente los pesos hacia valores pequeños. El segundo es una penalización L2 explícita y configurable. Durante el entrenamiento se añade a la pérdida un término proporcional a la suma de los cuadrados de los pesos del modelo. Esta penalización se aplica solo a las matrices de pesos, excluyendo los términos independientes y los parámetros de las capas de normalización, que no conviene penalizar de la misma forma.

Un detalle importante de trazabilidad es que, aunque este término de regularización se suma a la pérdida que guía la actualización de los parámetros, la métrica registrada como `train_loss` conserva únicamente la entropía cruzada. De este modo, la pérdida de entrenamiento y la de validación miden lo mismo y son directamente comparables entre sí y entre experimentos.

El ritmo de aprendizaje (*learning rate*) determina cómo de grandes son los pasos con los que se actualizan los parámetros en cada iteración. Un valor demasiado grande puede

hacer que el entrenamiento se vuelva inestable; uno demasiado pequeño lo ralentiza. Para gestionarlo se combinaron dos mecanismos.

El primero es un calentamiento inicial (*warmup*). Durante las primeras épocas, el ritmo de aprendizaje no parte de su valor máximo, sino que crece de forma gradual, ya sea de manera lineal o siguiendo un perfil más suave, hasta alcanzar el valor objetivo. Empezar con pasos pequeños evita las grandes oscilaciones que pueden producirse al inicio, cuando los parámetros aún están lejos de una buena solución.

El segundo mecanismo actúa una vez terminado el calentamiento: el ritmo de aprendizaje se reduce automáticamente cuando la pérdida de validación deja de mejorar durante varias épocas seguidas (**ReduceLROnPlateau**). La idea es dar pasos más finos a medida que el modelo se acerca a una solución y ya no progresa con el ritmo anterior. La implementación contempla también, como alternativa, un descenso suave y continuo del ritmo de aprendizaje a lo largo del entrenamiento (*cosine decay*).

Finalmente, el estado del optimizador y del controlador del ritmo de aprendizaje se guarda junto con el mejor modelo en cada punto de control (*checkpoint*). Esto permite reanudar un entrenamiento interrumpido exactamente en las mismas condiciones en que se dejó, lo que resultó útil para los entrenamientos largos ejecutados en el servidor.

4.7 Métricas, diagnósticos y trazabilidad

Cada entrenamiento genera una carpeta dentro de **runs/**. En ella se guardan la configuración resuelta, el checkpoint del mejor modelo, el archivo **metrics.csv**, las curvas de pérdida y los diagnósticos. Esto fue fundamental para poder reconstruir qué se había probado y con qué parámetros.

En cada época se registran en **metrics.csv** varias métricas que resumen el comportamiento del modelo sobre el conjunto de entrenamiento y sobre el de validación. La primera es la pérdida (**train_loss** y **eval_loss**), que mide cómo de lejos queda la predicción de la respuesta correcta. Cuanto más baja es, mejor es el ajuste, y penaliza especialmente los fallos cometidos con mucha seguridad. La versión **train** se calcula sobre los datos de entrenamiento y la **eval** sobre los recortes de validación, que el modelo no ha visto durante el ajuste. A continuación está el acierto o *accuracy* top-1 (**train_accuracy** y **eval_accuracy**), que es la fracción de recortes en los que la clase más probable para el modelo coincide con la real; es la medida más directa de rendimiento, de modo que un valor de 0,96 indica que el modelo acierta la primera predicción en el 96 % de los casos. El acierto top-5 (**eval_top5_accuracy**) es una versión más permisiva; cuenta los recortes en los que la clase correcta está entre las cinco más probables, aunque no sea la primera, y sirve para ver si el modelo se acerca a la respuesta correcta cuando su primera elección falla. Por último, la confianza media en aciertos y en errores (**mean_confidence_correct** y **mean_confidence_wrong**) recoge la probabilidad que el modelo asigna a la clase que elige; al promediarla por separado según acierte o falle, se puede comprobar si está bien

calibrado o si se equivoca con demasiada seguridad.

Al terminar cada entrenamiento, a partir del mejor checkpoint se calculan además métricas por clase sobre el conjunto de validación. La precisión (*precision*) de una clase indica, de todos los recortes que el modelo asigna a ella, qué fracción le pertenece realmente, por lo que penaliza los falsos positivos. La exhaustividad (*recall*) indica, de todos los recortes que pertenecen de verdad a esa clase, qué fracción consigue recuperar el modelo, por lo que penaliza los que se le escapan. El F1 combina ambas en su media armónica, que da más peso al menor de los dos valores, de manera que solo es alto cuando la precisión y la exhaustividad lo son a la vez. Para resumir el rendimiento global se usa la media macro de estas tres métricas, es decir, el promedio simple sobre las 25 clases dando el mismo peso a cada una con independencia de cuántos recortes contenga, lo que evita que las clases más numerosas dominen el resultado.

Junto a estas métricas se generan otros diagnósticos, como la matriz de confusión, la distribución de predicciones, las principales confusiones, las curvas *precision-recall* y las curvas F1-confianza. La matriz de confusión es especialmente útil, ya que sus filas representan la clase real y sus columnas la clase predicha, de modo que la diagonal recoge los aciertos y los valores fuera de ella muestran entre qué clases se confunde el modelo. En este TFG todas estas métricas se interpretan como métricas de clasificación de recortes, una distinción importante porque el paper de ChemEq25 evalúa modelos de detección, mientras que aquí el modelo clasifica recortes ya generados a partir de las anotaciones, sin localizar cajas ni medir solapamientos.

4.8 Comparación con ResNet50

Para situar el rendimiento de la arquitectura tensorial dentro de un marco más amplio, se incorporó como referencia una arquitectura convolucional estándar de visión artificial; ResNet50. El objetivo de esta comparación no es demostrar que un modelo supere al otro, sino contextualizar los resultados de la red tensorial frente a un modelo ampliamente utilizado y bien comprendido, e identificar qué decisiones de diseño influyen más en el rendimiento.

La comparación se diseñó para ser justa, en el sentido de aislar el efecto de la arquitectura. ResNet50 clasifica exactamente los mismos recortes que la red tensorial. Se utilizan las mismas particiones de entrenamiento, validación y test, las mismas 25 clases de ChemEq25 y la misma normalización de las imágenes (con la media y la desviación típica de ImageNet que ya produce la preparación de datos). Además, ambos modelos comparten el mismo bucle de entrenamiento y evaluación y se valoran con las mismas métricas. De este modo, la principal diferencia controlada entre los experimentos es la arquitectura del modelo.

El modelo se tomó de la librería `torchvision`, sustituyendo únicamente su capa final de clasificación por una capa lineal hacia las 25 clases del problema (con la opción de añadir regularización por *dropout*). Sobre esta base se evaluaron dos esquemas de entrenamiento,

que se corresponden con dos formas distintas de inicializar los pesos del modelo:

- **Entrenamiento desde cero.** La red parte de pesos aleatorios y aprende únicamente a partir de los recortes de ChemEq25, sin ningún conocimiento previo. Sirve como referencia de cuánto puede aprender la arquitectura con los datos disponibles del problema.
- **Ajuste fino.** La red parte de pesos preentrenados sobre ImageNet, un gran conjunto de imágenes de uso general, y todos sus pesos se actualizan durante el entrenamiento, ajustando las características visuales generales aprendidas previamente al problema concreto de material de laboratorio. Constituye la referencia más habitual y fuerte de visión artificial.

En la implementación, ambos esquemas se controlan con una opción de configuración que indica si se parte de pesos preentrenados sobre ImageNet o de pesos aleatorios. Cada esquema se entrenó con una tasa de aprendizaje distinta según su inicialización. Para el modelo entrenado desde cero se empleó un valor mayor, mientras que para el modelo preentrenado se utilizó un valor menor durante el ajuste fino, con el objetivo de adaptar sus pesos al nuevo problema sin modificar bruscamente las características ya aprendidas. Estos experimentos de referencia incorporaron además un criterio de parada temprana (*early stopping*), que detiene el entrenamiento cuando la pérdida de validación deja de mejorar durante un número fijo de épocas, evitando épocas innecesarias una vez alcanzado el mejor rendimiento. Los resultados cuantitativos de cada esquema, junto con su comparación con la red tensorial propuesta, se presentan y discuten en el capítulo de resultados.

4.9 Reproducibilidad y ejecución en servidor

Todos los entrenamientos se ejecutaron en los servidores Lorca de la universidad. El uso de estos recursos forma parte de la metodología del proyecto, porque permitió lanzar experimentos largos, aprovechar la aceleración por GPU, guardar puntos de control y analizar las métricas con posterioridad.

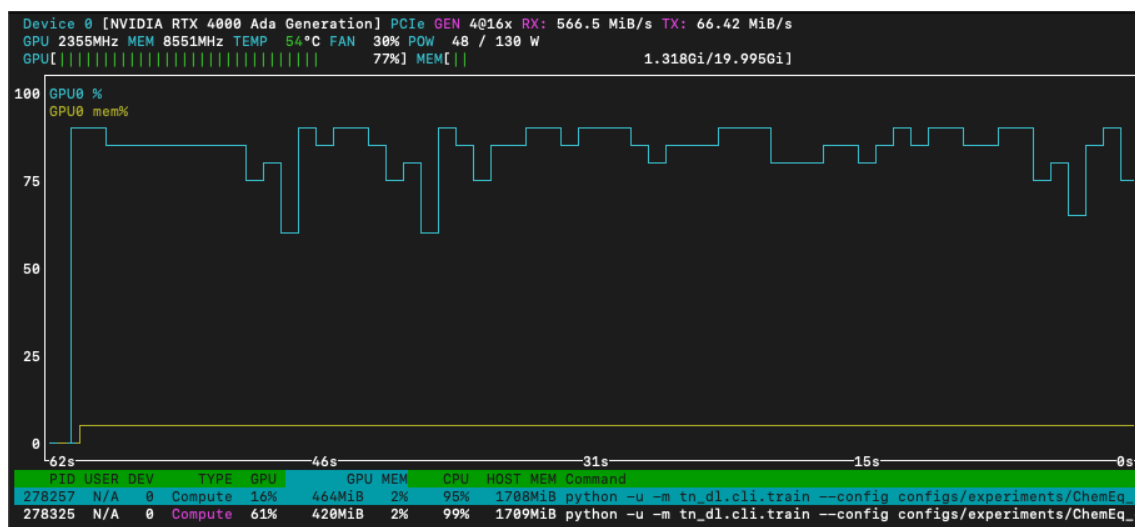


Figura 4.3: Captura de una GPU NVIDIA RTX 4000 Ada Generation del servidor Lorca de la UEM ejecutando dos procesos de entrenamiento en paralelo.

El trabajo en el servidor obligó a cuidar el entorno de Python, las dependencias y los scripts de lanzamiento, así como a registrar de forma ordenada los resultados de cada ejecución. Esto fue especialmente importante porque muchas pruebas se lanzaron en paralelo o en días distintos; sin un registro fiable habría sido difícil distinguir si una mejora provenía de la arquitectura, del preprocesamiento, de la semilla aleatoria, del ritmo de aprendizaje o de la regularización.

La reproducibilidad se cuidó de forma explícita. Cada ejecución fija una semilla aleatoria, lo que hace que el entrenamiento sea repetible, y conserva la configuración resuelta y las métricas ya descritas, de modo que un experimento puede reconstruirse a partir de esos archivos sin depender de notas externas. El código completo del proyecto está disponible en un repositorio público enlazado en los anexos, donde se incluye también una nota con las instrucciones necesarias para reproducir los experimentos.

Capítulo 5. RESULTADOS

En este capítulo se presentan los resultados obtenidos con el modelo tensorial desarrollado en el trabajo y se comparan con los de ResNet50, una arquitectura de red neuronal muy utilizada como referencia en tareas de clasificación de imágenes. El objetivo de esta comparación es situar el rendimiento del modelo tensorial dentro de un marco reconocible y comprobar hasta qué punto una arquitectura basada en *tensor networks* puede resolver de forma competitiva un problema real de clasificación.

Antes de analizar las cifras, conviene precisar qué se está evaluando. El conjunto ChemEq25 fue publicado originalmente para estudiar un problema de detección de materiales de laboratorio dentro de imágenes completas. En ese tipo de tarea, el modelo no solo debe decir qué objeto aparece, sino también localizar en qué zona de la imagen se encuentra. Por eso, los trabajos de detección suelen usar métricas que combinan la calidad de la localización y la calidad de la clasificación.

En este TFG se trabaja con una versión reformulada del problema. A partir de las anotaciones originales se generan recortes, cada uno de los cuales contiene un único objeto de laboratorio. Por tanto, el modelo ya no tiene que encontrar dónde está el objeto dentro de la imagen completa, sino únicamente decidir a cuál de las 25 clases pertenece el recorte. Dicho de otra forma, se evalúa un problema de clasificación de imágenes, no un problema completo de detección de objetos.

Esta distinción es importante para interpretar correctamente los resultados. Las métricas utilizadas en este capítulo, como *accuracy*, top-5, precisión, exhaustividad (*recall*) y F1, miden la calidad del clasificador sobre recortes individuales. Por ello, no son directamente comparables con las métricas de detección del artículo original de ChemEq25, como el *mAP@50*, que también tiene en cuenta si el objeto se ha localizado bien en la imagen [13]. Sin embargo, sí permiten comparar de forma justa el modelo tensorial propuesto con ResNet50, ya que ambos se entrenan y se evalúan sobre los mismos recortes, las mismas particiones de datos y las mismas 25 clases.

5.1 Mejor modelo tensorial obtenido

El mejor resultado obtenido corresponde a una arquitectura formada por tres partes: una etapa inicial de representación de la imagen, implementada mediante `AIMPatchEmbedding`; una red tensorial jerárquica, implementada como `HybridCustomTTN`; y una cabeza final de clasificación lineal. Esta combinación es la configuración final del modelo tensorial propuesto en el TFG.

Cada recorte de entrada se redimensiona a 224×224 píxeles y se transforma en una

representación discreta de 14×14 posiciones de entrada. Por tanto, cada imagen queda descrita por una secuencia de 196 vectores, que es la entrada que recibe la red tensorial. En esta configuración se fijaron `local_dim` = 12 y `bond_dim` = 12. La primera magnitud corresponde a la dimensión de cada vector local producido por el *embedding*, mientras que la segunda controla la dimensión interna de los enlaces de la red tensorial.

El entrenamiento se realizó con entropía cruzada como función de pérdida y el optimizador AdamW. Además, se empleó un *learning rate* inicial de 0,00025, una fase inicial de calentamiento (*warmup*) de tipo coseno durante 5 épocas, suavizado de etiquetas (*label smoothing*) de 0,05, regularización L_2 suave, *weight decay* en el optimizador, preservación de la relación de aspecto de los recortes y aumento de datos suave. Las métricas principales obtenidas sobre el conjunto de evaluación se recogen en [Tabla 5.1](#).

Métrica	Valor
Accuracy top-1	96,05 %
Accuracy top-5	99,43 %
Precisión macro	96,12 %
Exhaustividad (<i>recall</i>) macro	95,82 %
F1 macro	95,94 %
F1 ponderado	96,05 %
<code>classification_mAP</code>	98,41 %
<i>Eval loss</i> final	0,2037

Tabla 5.1: Métricas principales del mejor modelo tensorial sobre ChemEq25.

El modelo alcanza una *accuracy* top-1 del 96,05 %, lo que significa que clasifica correctamente algo más del 96 % de los recortes del conjunto de evaluación. Además, obtiene una *accuracy* top-5 del 99,43 %, de modo que, incluso cuando la primera predicción no es correcta, la clase verdadera aparece casi siempre entre las cinco clases más probables. Este comportamiento es relevante porque indica que los errores no suelen corresponder a predicciones completamente alejadas de la clase real, sino a confusiones entre clases que el modelo considera cercanas.

La precisión macro, la exhaustividad macro y el F1 macro se sitúan también en torno al 96 %. Estas métricas calculan primero el rendimiento clase a clase y después promedian los resultados, por lo que son útiles para comprobar que el buen rendimiento global no se debe únicamente a las clases más representadas. El F1 ponderado coincide prácticamente con la *accuracy*, lo que sugiere que el comportamiento agregado del modelo es coherente con el rendimiento observado por clase.

También se ha incluido el valor de `classification_mAP`, que alcanza el 98,41 %. En este caso, la métrica se calcula como una media de precisión clase a clase a partir de las probabilidades de salida del clasificador. Para ello, cada clase se evalúa por separado frente al conjunto de las demás clases, y después se promedian los resultados. Por tanto, no debe confundirse con el *mAP@50* utilizado en detección de objetos, ya que aquí no

se evalúa la localización espacial del objeto. Su valor indica, no obstante, que el orden de probabilidades producido por el clasificador es muy bueno.

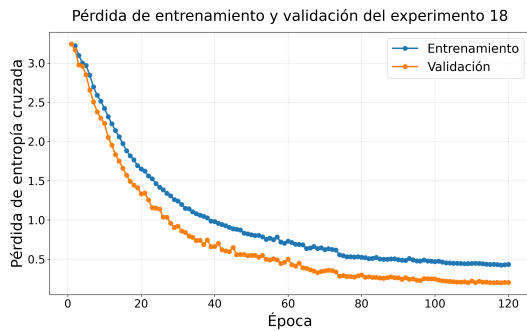
Un aspecto importante de este resultado es el tamaño del modelo. En este contexto, se llama parámetro entrenable a cada número interno del modelo que se ajusta durante el entrenamiento. Por ejemplo, son parámetros los coeficientes de los filtros convolucionales, los pesos de las capas lineales y las entradas de los tensores que forman la red tensorial. No se cuentan como parámetros los datos de entrada, las clases del problema, las dimensiones elegidas como hiperparámetros ni los nodos de relleno fijos introducidos para completar la estructura de la TTN.

El recuento se realizó sumando todos los parámetros entrenables de la arquitectura una vez construida con la configuración final. La arquitectura tensorial completa tiene aproximadamente 2,15 millones de parámetros entrenables. La mayor parte de ellos se concentra en los bloques MPS-3 que forman los distintos niveles de la `HybridCustomTTN`, con alrededor de 2,10 millones de parámetros. La etapa inicial `AIMPatchEmbedding` aporta aproximadamente 48 000 parámetros, mientras que la cabeza lineal final contiene únicamente unos cientos. Este recuento será especialmente relevante en la comparación posterior con ResNet50, cuyo número de parámetros es considerablemente mayor.

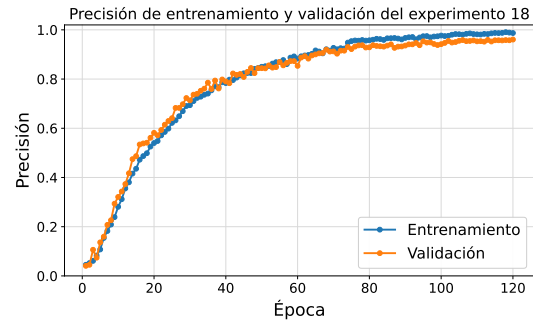
5.2 Comportamiento del modelo tensorial

Además de las métricas finales, es importante analizar cómo se comporta el modelo durante el entrenamiento. Esto permite comprobar si el resultado obtenido es estable, si el modelo aprende de forma progresiva y si existe una separación excesiva entre el rendimiento sobre los datos de entrenamiento y el rendimiento sobre datos no vistos.

En la [Figura 5.1](#) se muestra la evolución del entrenamiento del mejor modelo tensorial. La función de pérdida mide, de forma agregada, cuánto se equivoca el modelo al asignar probabilidades a las distintas clases. Por tanto, una disminución de esta magnitud indica que el modelo está ajustando mejor sus predicciones. La precisión, por otro lado, mide directamente el porcentaje de recortes clasificados correctamente.



(a) Evolución de la pérdida.



(b) Evolución de la precisión.

Figura 5.1: Evolución del entrenamiento del mejor modelo tensorial. En (a) se representa la función de pérdida y en (b) la precisión, comparando el comportamiento sobre entrenamiento y evaluación.

La evolución observada muestra que el modelo aprende de forma progresiva durante el entrenamiento. La precisión sobre el conjunto de evaluación aumenta de manera sostenida hasta alcanzar un valor máximo del 96,05 %, mientras que la pérdida de evaluación desciende hasta un valor final de 0,2037. Esto indica que el modelo no solo mejora sobre los datos usados para ajustar sus parámetros, sino que también mantiene un buen rendimiento sobre ejemplos no utilizados directamente durante el entrenamiento.

La comparación entre entrenamiento y evaluación permite estudiar si el modelo está memorizando en exceso los datos de entrenamiento. En este caso, el comportamiento general no apunta a un sobreajuste severo, porque el rendimiento sobre el conjunto de evaluación se mantiene alto y no se observa una degradación clara al final del entrenamiento. Aun así, la diferencia entre el mejor resultado y otros entrenamientos equivalentes muestra que el modelo sí presenta cierta sensibilidad a la inicialización aleatoria de sus parámetros.

Para comprobar esta variabilidad, se repitió el entrenamiento manteniendo la misma configuración y cambiando únicamente la semilla aleatoria. Esta semilla controla aspectos como la inicialización inicial de los parámetros y el orden en el que se presentan los datos durante el entrenamiento. Los valores de *accuracy* obtenidos fueron 94,54 %, 94,19 %, 96,05 % y 94,26 %, con una media de 94,76 %.

Este resultado matiza la interpretación del mejor modelo. El valor de 96,05 % representa el mejor entrenamiento obtenido, pero el comportamiento más habitual de la arquitectura se sitúa en torno al 94–95 %. Por tanto, el modelo tensorial es capaz de alcanzar un rendimiento alto, aunque todavía muestra una dependencia apreciable de las condiciones iniciales del entrenamiento. Esta sensibilidad es una limitación relevante, especialmente si se compara con arquitecturas más consolidadas en visión artificial.

También resulta útil analizar qué tipo de errores comete el modelo. Para ello se estudió la matriz de confusión, que recoge cuántas veces cada clase real fue asignada a cada clase predicha. Esta representación permite distinguir entre errores aislados y confusiones sistemáticas entre clases concretas.

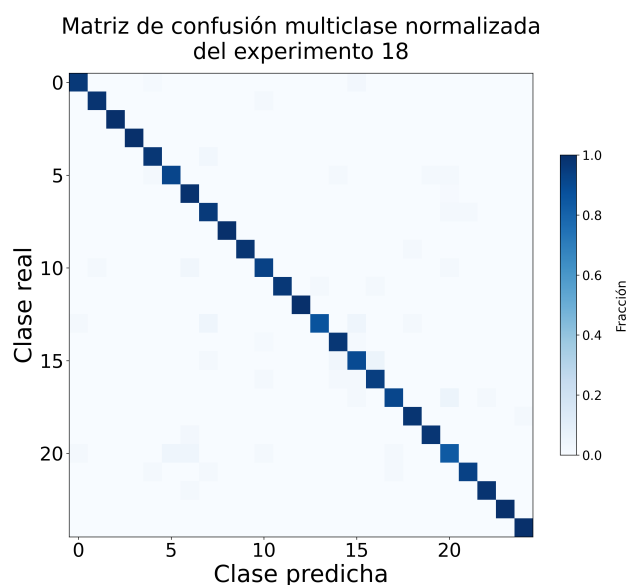


Figura 5.2: Matriz de confusión del mejor modelo tensorial sobre el conjunto de evaluación. La concentración de valores en la diagonal principal indica que la mayoría de los recortes se clasifican correctamente.

En conjunto, la matriz de confusión muestra un comportamiento muy concentrado en la diagonal principal, lo que significa que la mayoría de los recortes se clasifican correctamente. Los errores aparecen principalmente entre clases visualmente parecidas o entre objetos cuya forma puede resultar ambigua cuando se observan como recortes individuales. Este efecto es razonable en el contexto de ChemEq25, ya que varias clases corresponden a material de laboratorio con geometrías similares, objetos alargados o recipientes transparentes, donde la información de contexto puede ser importante.

Esta lectura también ayuda a interpretar la diferencia entre la precisión top-1 y la precisión top-5. Aunque el modelo se equivoca en torno al 4 % de los casos cuando solo se considera la clase más probable, la clase correcta aparece entre las cinco opciones más probables en el 99,43 % de los recortes. Por tanto, muchos errores no son predicciones completamente incoherentes, sino confusiones entre clases que el modelo considera cercanas. Esto sugiere que la representación aprendida por la red tensorial organiza razonablemente bien las clases, aunque no siempre separa con suficiente claridad aquellas que son visualmente similares.

5.3 Comparación con ResNet50

Para contextualizar el rendimiento del modelo tensorial, se comparó con ResNet50, una arquitectura de red neuronal convolucional ampliamente utilizada como referencia en ta-

reas de clasificación de imágenes. A diferencia de la red tensorial propuesta, ResNet50 está diseñada específicamente para procesar imágenes mediante convoluciones, es decir, operaciones locales que extraen patrones espaciales como bordes, formas y texturas. Por ello, no se espera necesariamente que el modelo tensorial supere a ResNet50, sino que la comparación sirve para situar su rendimiento frente a una arquitectura estándar y reconocible en visión artificial.

La comparación se realizó en las mismas condiciones de datos. Los tres modelos se entrenaron y evaluaron sobre los mismos recortes de ChemEq25, con las mismas 25 clases y las mismas particiones de entrenamiento, validación y evaluación. De esta forma, la diferencia entre modelos se debe principalmente a la arquitectura utilizada y no a cambios en el conjunto de datos.

Se evaluaron dos versiones de ResNet50. En la primera, todos los parámetros se inicializan de forma aleatoria y el modelo aprende únicamente a partir de ChemEq25. A este caso se le denomina entrenamiento desde cero. En la segunda, se parte de una ResNet50 que ya había sido entrenada previamente sobre ImageNet, un conjunto de imágenes mucho más grande y variado, y después se ajustan sus parámetros al problema concreto de ChemEq25. Este procedimiento se conoce como ajuste fino o *fine-tuning*. La diferencia entre ambos casos es que el modelo entrenado desde cero solo utiliza la información disponible en ChemEq25, mientras que el modelo con ajuste fino aprovecha representaciones visuales aprendidas previamente en otro conjunto de datos.

Los resultados de la comparación se recogen en [Tabla 5.2](#). Además de la precisión top-1 y top-5, se incluye el número aproximado de parámetros entrenables de cada modelo. La precisión top-1 corresponde al porcentaje de recortes en los que la clase más probable predicha por el modelo coincide con la clase real. La precisión top-5 considera correcta la predicción si la clase real aparece entre las cinco clases a las que el modelo asigna mayor probabilidad. En la tabla se muestran los mejores valores registrados para cada modelo, ya que el objetivo es comparar el rendimiento alcanzado por cada arquitectura y no la evolución época a época.

Modelo	Top-1	Top-5	Parámetros	Observación
AIM + HybridCustomTTN	96,05 %	99,43 %	$\approx 2,15$ M	Modelo tensorial propuesto
ResNet50 desde cero	97,27 %	99,64 %	23,56 M	Sin pesos previos
ResNet50 con ajuste fino	99,71 %	99,86 %	23,56 M	Con pesos preentrenados

Tabla 5.2: Comparación entre el modelo tensorial propuesto y ResNet50 sobre los mismos recortes de ChemEq25.

La tabla anterior resume las diferencias principales entre modelos. Para visualizar mejor la comparación, en la [Figura 5.3](#) se muestran por separado el rendimiento en clasificación y el tamaño aproximado de cada arquitectura.

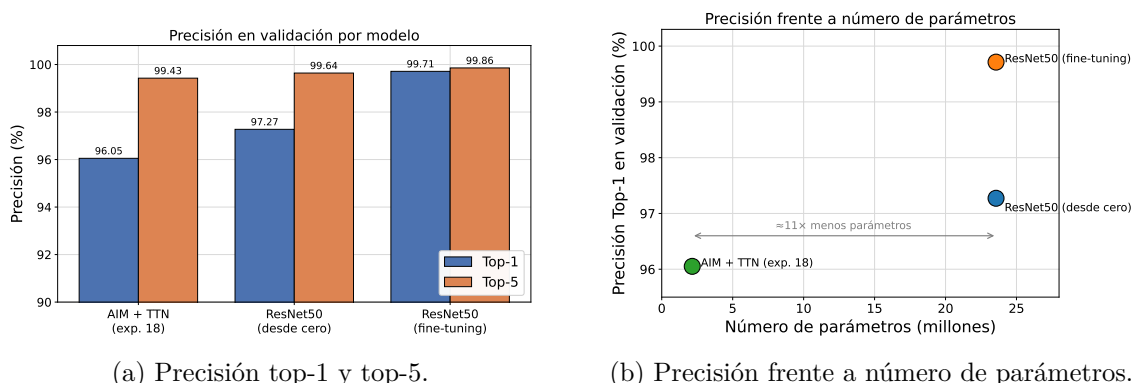


Figura 5.3: Comparación entre el modelo tensorial propuesto y ResNet50. En (a) se muestra la precisión top-1 y top-5 alcanzada por cada modelo y en (b) la relación entre la precisión top-1 y el número aproximado de parámetros entrenables.

El mejor resultado absoluto lo obtiene ResNet50 con ajuste fino, alcanzando una precisión top-1 del 99,71 % y una precisión top-5 del 99,86 %. Este resultado es coherente con lo esperado, ya que el modelo, antes de adaptarse a ChemEq25, ya había aprendido representaciones visuales generales en ImageNet. Por tanto, esta comparación no mide únicamente la capacidad de ResNet50 sobre el conjunto ChemEq25, sino también la ventaja que proporciona el aprendizaje previo en un conjunto de datos mucho mayor.

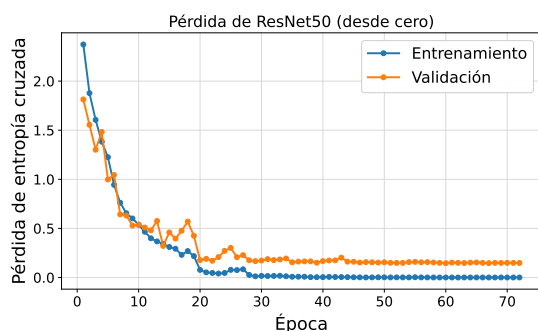
La comparación más directa para valorar la arquitectura tensorial es la realizada frente a ResNet50 entrenada desde cero. En este caso, ResNet50 alcanza una precisión top-1 del 97,27 %, mientras que el modelo tensorial obtiene un 96,05 %. La diferencia es de aproximadamente 1,2 puntos porcentuales, por lo que ResNet50 sigue siendo superior en rendimiento absoluto, pero la distancia no es grande. Esto indica que la arquitectura tensorial propuesta es capaz de resolver el problema con una precisión cercana a la de una red convolucional profunda entrenada específicamente sobre los mismos datos.

La diferencia principal aparece al comparar el tamaño de los modelos. ResNet50 tiene aproximadamente 23,56 millones de parámetros, mientras que el modelo tensorial completo tiene alrededor de 2,15 millones. Es decir, la red tensorial utiliza unas once veces menos parámetros que ResNet50. Desde este punto de vista, el resultado es favorable para la propuesta del TFG. Aunque no alcanza la precisión de ResNet50, consigue un rendimiento competitivo con un modelo mucho más pequeño.

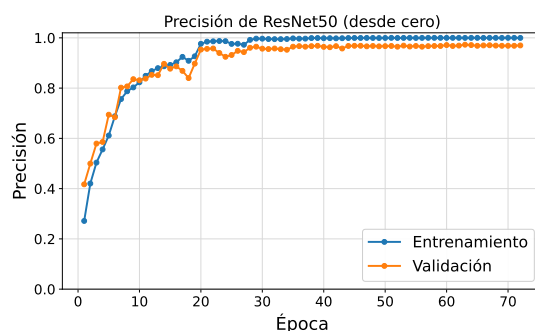
Este punto es especialmente relevante porque una de las motivaciones de las *tensor networks* en aprendizaje automático es explorar arquitecturas con una estructura interna más controlada y potencialmente más eficiente en número de parámetros. En este experimento, esa ventaja sí aparece de forma clara en el tamaño del modelo. No obstante, tener menos parámetros no implica automáticamente que el modelo sea mejor en todos los sentidos. También importan otros factores, como el tiempo de entrenamiento, la facilidad de implementación, la estabilidad frente a distintas inicializaciones y la madurez de las herramientas disponibles.

En conjunto, la comparación muestra una conclusión equilibrada. ResNet50, especialmente cuando parte de pesos preentrenados, sigue siendo la opción más potente para este problema de clasificación. Sin embargo, el modelo tensorial desarrollado alcanza una precisión alta y se aproxima al rendimiento de ResNet50 entrenada desde cero utilizando un número de parámetros considerablemente menor. Por tanto, el resultado no demuestra que las redes tensoriales sustituyan a las arquitecturas convolucionales estándar, pero sí muestra que pueden constituir una alternativa competitiva y técnicamente interesante para problemas reales de clasificación de imágenes.

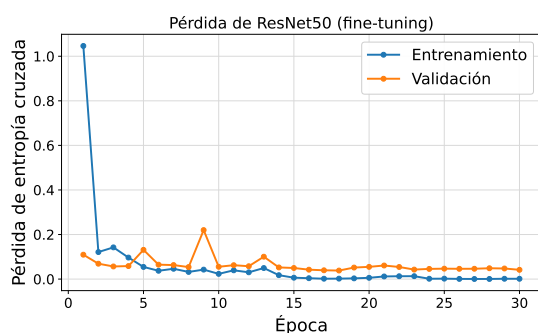
Finalmente, en la [Figura 5.4](#) se muestra la evolución del entrenamiento de las dos versiones de ResNet50, tanto en términos de pérdida como de precisión. En el caso entrenado desde cero (figuras 5.4a y 5.4b), la pérdida de entrenamiento descende de forma progresiva hasta valores muy próximos a cero, mientras que la pérdida de validación se estabiliza en torno a 0,15; esta separación entre ambas curvas refleja un cierto grado de sobreajuste, esperable al entrenar una red profunda desde cero sobre un conjunto de datos relativamente pequeño. Aun así, la precisión de validación se mantiene estable en torno al 97 %, lo que indica que el modelo generaliza correctamente. En el caso con ajuste fino (figuras 5.4c y 5.4d), la convergencia es mucho más rápida, ya que en apenas unas pocas épocas el modelo alcanza una precisión de validación cercana al 99,7 %, lo que evidencia la ventaja de partir de representaciones visuales ya aprendidas en ImageNet.



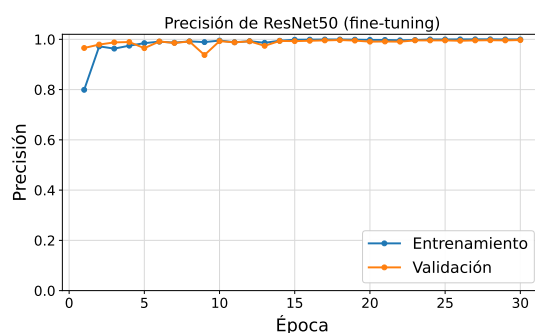
(a) Pérdida, desde cero.



(b) Precisión, desde cero.



(c) Pérdida, ajuste fino.



(d) Precisión, ajuste fino.

Figura 5.4: Evolución del entrenamiento de ResNet50 sobre ChemEq25. Fila superior: modelo entrenado desde cero. Fila inferior: modelo con ajuste fino a partir de pesos pre-entrenados en ImageNet.

Capítulo 6. CONCLUSIONES

6.1 Conclusiones del trabajo

El modelo tensorial desarrollado alcanza, en su mejor entrenamiento, un 96,05 % de *accuracy*, un 95,94 % de F1 macro y un 99,43 % de *top-5 accuracy* sobre ChemEq25. Es una cifra alta para una arquitectura basada en una *Tree Tensor Network*. Esta sección se centra en interpretar de forma crítica ese resultado y sus límites.

La observación más importante que arrojan los experimentos es la inestabilidad del modelo frente a la inicialización. Al repetir el mejor experimento cambiando únicamente la semilla aleatoria, las *accuracies* obtenidas fueron 94,54 %, 94,19 %, 96,05 % y 94,26 %, con una media de 94,76 %. El 96,05 % es, por tanto, un valor atípico por encima de la media, y no la cifra que cabría esperar de una ejecución cualquiera. El comportamiento típico de la arquitectura se sitúa en torno al 94–95 %. Esto obliga a matizar el resultado principal en dos sentidos. El primero es que presentar el mejor entrenamiento como rendimiento del modelo sobreestima su capacidad real en aproximadamente un punto porcentual. En segundo lugar, una varianza de este orden entre semillas indica que el proceso de optimización no converge de forma robusta a una misma solución, lo que es en sí mismo un resultado a tener en cuenta, ya que cualquier comparación honesta con otra arquitectura debería hacerse sobre la media de varias ejecuciones, no sobre el mejor caso.

La comparación con ResNet50 debe leerse bajo esa misma prudencia. ResNet50 entrenada desde cero alcanza un 97,27 % y, con ajuste fino a partir de pesos de ImageNet, un 99,71 %. Frente al mejor entrenamiento tensorial, la diferencia es de algo más de un punto respecto a ResNet50 desde cero; pero si se compara contra el rendimiento medio del modelo tensorial (94,76 %), la brecha se acerca a los dos puntos y medio, y crece hasta casi cinco puntos frente a la versión con *fine-tuning*. La conclusión es, por tanto, que el modelo tensorial se aproxima a una CNN con un margen moderado y con mayor variabilidad. Donde la arquitectura tensorial sí presenta una ventaja clara y cuantificable es en la economía de parámetros. Utiliza unos 2,15 millones de parámetros entrenables frente a los 23,56 millones de ResNet50, es decir, alrededor de once veces menos. Este es probablemente el resultado más valioso del trabajo, porque conecta directamente con la motivación de las *Tensor Networks* como forma estructurada y compacta de representar transformaciones. Aun así, un menor número de parámetros no se traduce automáticamente en un modelo mejor, ya que la comparación deja fuera el tiempo de entrenamiento, la estabilidad, el consumo de memoria durante las contracciones y la madurez del ecosistema de herramientas, aspectos en los que ResNet50 sigue siendo más ventajosa.

Un segundo hallazgo que merece una lectura crítica es el origen de las mejoras observadas a lo largo del desarrollo. Los saltos de rendimiento no vinieron de modificar la topolo-

gía tensorial ni de aumentar su capacidad; de hecho, ampliar la dimensión de enlace no garantizó una mejor generalización. El rendimiento mejoró gracias a la preservación de la relación de aspecto, el *label smoothing*, la regularización, una estrategia suave de *data augmentation* y también a la representación de entrada mediante **AIMPatchEmbedding**, que conserva la información espacial antes de entregarla a la TTN. Esto sugiere que buena parte del rendimiento final es atribuible al preprocesamiento y a técnicas estándar de *deep learning*, y no está claro cuánto aporta la estructura tensorial por sí sola. El trabajo no incluye un estudio de ablación que aísle la contribución de la TTN frente a la del *embedding* convolucional que la precede, por lo que esta cuestión queda abierta y constituye, seguramente, la limitación metodológica más relevante del proyecto.

Conviene además delimitar con precisión el alcance de lo que se ha medido. Todas las métricas se han calculado como clasificación de recortes ya extraídos a partir de las anotaciones, y no como detección; esto las distingue de las que reporta el propio trabajo de ChemEq25, que evalúa modelos de detección, de modo que las cifras no son directamente comparables con las de la literatura sobre ese conjunto. Asimismo, la evaluación se ha limitado a un único *dataset* y a una única arquitectura de referencia, ResNet50. No se han contrastado otras familias de modelos, como los *Transformers* de visión, ni se ha probado el enfoque en dominios distintos al del material de laboratorio.

En balance, el trabajo demuestra que una arquitectura basada en Tensor Networks puede integrarse en un *pipeline* de clasificación de imágenes y alcanzar un rendimiento cercano al de una CNN consolidada empleando un orden de magnitud bastante menor de parámetros. Los puntos débiles no invalidan la propuesta, pero sí acotan lo que puede afirmarse a partir de ella. De ellos se derivan las líneas de trabajo futuro más útiles; realizar experimentos que separen la contribución de la TTN de la del *embedding*, y ampliar la comparación a otras arquitecturas y conjuntos de datos para extraer conclusiones más generales sobre la competitividad de las Tensor Networks en visión artificial.

6.2 Conclusiones personales

El desarrollo de este Trabajo de Fin de Grado ha supuesto una experiencia especialmente enriquecedora, tanto desde el punto de vista académico como personal. En primer lugar, el tema abordado pertenece a un ámbito muy especializado, las *tensor networks*, cuyo funcionamiento requiere una comprensión profunda tanto de conceptos físicos como de herramientas computacionales avanzadas. Haber tenido la oportunidad de trabajar y de aprender bajo la supervisión de investigadores con experiencia en el área, en particular Alejandro Mata Ali y todo el departamento de computación cuántica del Instituto Tecnológico de Castilla y León, ha sido un privilegio y una parte fundamental del proceso formativo asociado a este trabajo.

A lo largo de estos meses, este TFG no solo me ha permitido profundizar en la física de las *tensor networks*, sino también adquirir una gran cantidad de competencias técnicas que han resultado esenciales para el desarrollo del proyecto. Entre ellas, destaca el aprendizaje

de Python a un nivel mucho más avanzado, así como la experiencia de programar desde cero un proyecto de cierta envergadura. Esto ha requerido aprender buenas prácticas de programación, para organizar y estructurar el trabajo de forma clara y mantenible.

Además, el proyecto ha supuesto un reto importante en el ámbito del *machine learning*. Ha sido necesario familiarizarse con PyTorch, estudiar las bases matemáticas del *deep learning* y comprender el flujo general de trabajo asociado a este tipo de modelos, desde la preparación de los datos hasta el entrenamiento y análisis de resultados. Todo ello ha ampliado considerablemente mi formación computacional y me ha permitido conectar herramientas modernas de inteligencia artificial con problemas propios de la física teórica y computacional.

Otro aspecto fundamental del desarrollo de este trabajo ha sido el aprendizaje de herramientas habituales en el entorno investigador. Durante el proyecto he aprendido a utilizar Git y GitHub para el control de versiones y la gestión del código, así como comandos básicos de Linux y Bash necesarios para trabajar de manera eficiente en los servidores de la universidad (Lorca). También he tenido la oportunidad de utilizar recursos de computación de alto rendimiento, que han sido clave para llevar a cabo los cálculos y entrenamientos necesarios. Esta experiencia me ha permitido comprender mejor cómo se organiza y ejecuta un proyecto computacional real en un entorno de investigación.

Más allá de los conocimientos técnicos, este TFG me ha acercado al funcionamiento cotidiano de la investigación científica. Haber podido participar en reuniones, observar la forma de trabajo del grupo y ver de primera mano cómo se plantean, discuten y desarrollan problemas de investigación ha sido una parte especialmente valiosa de la experiencia. En este sentido, el trabajo no solo ha consistido en resolver un problema concreto, sino también en aprender cómo se investiga, cómo se colabora y cómo se construye conocimiento en un contexto académico real.

Este TFG ha representado un desafío constante. Por un lado, debido a la dificultad conceptual asociada a la física de las *tensor networks*; por otro, por la exigencia técnica derivada de la programación, el uso de herramientas de aprendizaje automático y el trabajo en entornos computacionales avanzados. Sin embargo, precisamente esta combinación de retos ha hecho que el proceso sea especialmente formativo. En apenas unos meses he adquirido conocimientos y habilidades que al inicio del proyecto no imaginaba poder desarrollar, y considero que esta experiencia ha supuesto un punto de inflexión importante en mi formación como físico.

En definitiva, ha sido un proyecto muy ambicioso que ha puesto a prueba todos mis conocimientos sobre física aprendidos en la carrera y ha ampliado considerablemente esos conocimientos, más especializados en la física computacional. También me ha permitido crecer a nivel científico, ya que he aprendido muchas herramientas que me servirán en mi carrera profesional.

GLOSARIO

Este glosario reúne los principales términos empleados a lo largo del trabajo. Se divide en tres bloques: un listado de acrónimos y siglas, un glosario de conceptos teóricos relacionados con las redes tensoriales y el aprendizaje profundo, y un glosario de los términos específicos de la implementación y del código desarrollado. Las definiciones de este último bloque se han redactado a partir del código fuente del proyecto y describen el comportamiento real de cada componente.

Acrónimos y siglas

AIM *Antisymmetric Interaction Module*. Módulo de interacción que combina dos ramas de operaciones aplicadas en orden inverso mediante un producto elemento a elemento, introduciendo una interacción multiplicativa (bilineal) entre las características de la imagen.

AdamW *Adam with decoupled Weight decay*. Variante del optimizador Adam que incorpora de forma desacoplada la penalización por reducción de pesos (*weight decay*).

BatchNorm *Batch Normalization*. Normalización por lotes: reescala las activaciones usando estadísticas del propio lote para estabilizar el entrenamiento.

CNN *Convolutional Neural Network*. Red neuronal convolucional, arquitectura clásica de visión artificial basada en la localidad espacial y en pesos compartidos.

CP *Canonical Polyadic*. Descomposición tensorial que expresa un tensor como suma de tensores de rango 1.

DMRG *Density Matrix Renormalization Group*. Grupo de renormalización de la matriz densidad, algoritmo de referencia en física de muchos cuerpos, estrechamente ligado a las MPS.

DTTN *Deep Tree Tensor Network*. Red tensorial jerárquica profunda; el trabajo del que se inspira el bloque AIM empleado en el *embedding*.

ChemEq25 *Chemistry Lab Apparatus dataset*. Conjunto de datos de imágenes reales de material de laboratorio, con 25 clases, utilizado como dataset principal del trabajo.

F1 Media armónica de la precisión y la exhaustividad (*recall*); métrica de clasificación por clase.

GPU *Graphics Processing Unit*. Unidad de procesamiento gráfico, empleada para acelerar el entrenamiento de los modelos.

HOSVD *Higher-Order Singular Value Decomposition*. Descomposición en valores singulares de orden superior; caso ortogonal de la descomposición de Tucker.

HPC *High-Performance Computing*. Computación de altas prestaciones; en este trabajo, los servidores Lorca de la UEM.

IA Inteligencia artificial.

ITCL Instituto Tecnológico de Castilla y León.

MERA *Multi-scale Entanglement Renormalization Ansatz*. Red tensorial jerárquica con desentrelazadores que representa correlaciones a todas las escalas de longitud.

MLP *Multi-Layer Perceptron*. Perceptrón multicapa; opción de cabeza de clasificación con una capa oculta.

MNIST *Modified National Institute of Standards and Technology*. Conjunto de datos de dígitos manuscritos, referencia clásica en clasificación de imágenes.

MPS *Matrix Product State*. Estado de producto de matrices; topología de red tensorial en cadena lineal, equivalente al formato Tensor Train.

PEPS *Projected Entangled Pair States*. Generalización bidimensional de la MPS, en la que cada tensor se conecta con cuatro vecinos.

ReLU *Rectified Linear Unit*. Unidad lineal rectificada, función de activación $\sigma(z) = \max(0, z)$.

ResNet50 *Residual Network* de 50 capas. Arquitectura convolucional estándar empleada como modelo de referencia.

RGB *Red, Green, Blue*. Los tres canales de color de una imagen.

SVD *Singular Value Decomposition*. Descomposición en valores singulares; herramienta algebraica fundamental de las redes tensoriales.

TFG Trabajo de Fin de Grado.

TN *Tensor Network*. Red tensorial; representación factorizada de un tensor de alto orden mediante tensores de menor orden conectados.

TT *Tensor Train*. Tren de tensores; nombre del formato MPS en el contexto del análisis numérico.

TTN *Tree Tensor Network*. Red tensorial en forma de árbol; topología jerárquica empleada en la arquitectura propuesta.

Tucker Descomposición tensorial que introduce un tensor núcleo que acopla varias matrices de factores.

UEM Universidad Europea de Madrid.

YAML *Yet Another Markup Language*. Formato de texto empleado para los archivos de configuración de los experimentos.

YOLO *You Only Look Once*. Familia de modelos de detección de objetos; en este trabajo se refiere al formato de sus anotaciones.

Conceptos teóricos

Tensor Objeto matemático de orden N cuyos elementos poseen N índices; generaliza los conceptos de escalar (orden 0), vector (orden 1) y matriz (orden 2) a dimensiones arbitrarias. Desde el punto de vista computacional, un *array* multidimensional.

Orden (o modo) de un tensor Número de índices del tensor, es decir, número de dimensiones que posee. Cada modo tiene una longitud asociada.

Contracción tensorial Operación por la cual dos (o más) tensores se combinan sumando sobre uno o varios pares de índices compartidos. Al contraer un tensor de orden n con uno de orden m a lo largo de q pares de índices, el resultado tiene orden $n + m - 2q$. Generaliza el producto matricial y el producto escalar.

Traza Contracción de una matriz consigo misma a través de un par de índices repetidos, $\text{Tr}(\mathbf{A}) = \sum_i A_{ii}$.

Producto exterior Operación (denotada $\circ$) que combina vectores para formar un tensor de rango 1; base de la descomposición CP.

Notación diagramática de Penrose Representación visual de los tensores como nodos con tantas aristas como índices. Las aristas unidas representan índices contraídos y las aristas libres, los índices del tensor resultante. El número de aristas libres es el orden del tensor final.

Red tensorial (Tensor Network) Representación factorizada de un tensor de alto orden como una colección de tensores de menor orden conectados por índices compartidos, en lugar de almacenar explícitamente todos sus elementos.

Índice físico (o de entrada) Índice que conecta la red tensorial con los datos. Queda libre tras las contracciones y corresponde a una posición de entrada, característica o salida del modelo.

Índice virtual (o de enlace) Índice interno que conecta dos tensores de la red y que se suma durante la contracción; no aparece en el tensor final, pero controla cuánta información puede transmitirse entre las distintas partes de la red.

Dimensión de enlace (*bond dimension*) Dimensión de los índices virtuales. Es el hiperparámetro principal que regula el compromiso entre compacidad (pocos parámetros) y capacidad de representar correlaciones complejas.

Maldición de la dimensionalidad Crecimiento exponencial (d^N) del número de componentes de un tensor de orden N con índices de dimensión d , que hace inviable su almacenamiento y manipulación directos. Las redes tensoriales surgen para mitigar este problema.

Entrelazamiento En física cuántica de muchos cuerpos, medida de las correlaciones entre partes de un sistema. Los estados con entrelazamiento limitado admiten representaciones tensoriales compactas.

Mapa local de características (*feature map*) Transformación $\phi(x)$ que lleva cada posición de entrada a un vector de dimensión local d . El producto tensorial de estos vectores define la representación de la entrada que se contrae con la red tensorial.

Descomposición en valores singulares (SVD) Factorización $\mathbf{A} = \mathbf{U}\mathbf{\Sigma}\mathbf{V}^\top$ de una matriz. Permite obtener la mejor aproximación de bajo rango en norma de Frobenius y es la herramienta básica para construir formatos como el Tensor Train.

Teorema de Eckart–Young Resultado que garantiza que, al conservar solo los mayores valores singulares de una matriz, se obtiene su mejor aproximación de rango reducido.

Descomposición CP Aproximación de un tensor como suma de R tensores de rango 1. Muy compacta, pero determinar el rango CP es un problema NP-difícil.

Descomposición de Tucker Factorización de un tensor mediante un tensor núcleo que acopla varias matrices de factores. Más flexible que CP, pero su núcleo crece exponencialmente con el orden.

Tensor Train / Matrix Product State (MPS) Representación de un tensor de orden N como una cadena de tensores locales conectados por índices virtuales. Su número de parámetros crece linealmente con N si los rangos se mantienen acotados. Es la topología de red tensorial más estudiada.

Tree Tensor Network (TTN) Topología jerárquica en forma de árbol: las hojas se asocian a las posiciones de entrada y se contraen por grupos hacia niveles superiores hasta un único tensor raíz. La distancia efectiva entre dos hojas crece como $\mathcal{O}(\log N)$, lo que favorece las correlaciones de largo alcance frente a la MPS.

Hoja (*leaf*) Tensor del nivel más bajo de una TTN, asociado directamente a una posición de entrada.

Tensor raíz Único tensor del nivel superior de una TTN, cuya salida resume toda la entrada y sobre la que se realiza la clasificación.

MERA Red tensorial jerárquica que, además de las capas de renormalización (isometrías) de una TTN, introduce *disentanglers* para eliminar el entrelazamiento de corto alcance, permitiendo representar correlaciones a todas las escalas.

Disentangler (desentrelazador) Tensor de la MERA que actúa sobre pares de posiciones vecinas para eliminar el entrelazamiento de corto alcance antes de la renormalización.

PEPS Generalización bidimensional de la MPS en la que cada tensor se conecta con cuatro vecinos, reflejando la geometría de una rejilla. Su contracción exacta es $\#P$ -difícil.

Grupo de renormalización Marco de la física en el que cada capa jerárquica describe el sistema a una escala de longitud distinta; interpretación natural de las capas de la MERA.

Linealización (recorrido en zigzag) Ordenación de las posiciones de una imagen bidimensional en una secuencia unidimensional. El recorrido en zigzag alterna el sentido de recorrido en filas consecutivas para preservar en parte la vecindad espacial.

Sesgo inductivo Conjunto de suposiciones que una arquitectura incorpora sobre la estructura de los datos; por ejemplo, la localidad espacial y los pesos compartidos de las convoluciones.

Logits Puntuaciones por clase sin normalizar que produce el modelo antes de convertirse en probabilidades. Se comparan con la etiqueta real mediante la función de pérdida.

Términos de implementación y código

Las definiciones de este bloque describen componentes concretos del código desarrollado para el trabajo (paquete `tn_dl`). Los nombres en tipografía monoespaciada corresponden a clases, funciones o hiperparámetros tal y como aparecen en el proyecto.

Componentes de la arquitectura

AIMPatchEmbedding Clase del *embedding* definitivo del proyecto. Transforma una imagen $[B, C, H, W]$ en una secuencia $[B, L, \text{local_dim}]$ conservando la estructura espacial bidimensional hasta el último paso. Aplica un *stem* convolucional que reduce la resolución (para 224×224 y `patch_size = 16`: $224 \rightarrow 112 \rightarrow 56 \rightarrow 28 \rightarrow 14$), después `num_aim_blocks` bloques AIM2D, una convolución 1×1 que ajusta los canales a `local_dim` y, finalmente, un aplanado con recorrido en zigzag que produce las 196 posiciones de entrada.

AIM2D Implementación del *Antisymmetric Interaction Module* adaptado a imágenes. Recibe y devuelve un mapa $[B, C, H, W]$. Construye dos ramas que aplican las mismas operaciones (una transformación lineal sobre canales y una convolución) en orden inverso, las combina mediante un producto de Hadamard (obteniendo una interacción bilineal, sin activaciones ni sesgos intermedios), normaliza con *LayerNorm*, proyecta

de vuelta a los canales originales con una convolución 1×1 y suma el resultado a la entrada mediante una conexión residual con factor de escala entrenable.

HybridCustomTTN Clase que implementa la red tensorial jerárquica (TTN) propuesta. Construye niveles sucesivos (**TTNLevel**) que agrupan las posiciones de tres en tres, reduciéndolas nivel a nivel ($196 \rightarrow 66 \rightarrow 22 \rightarrow 8 \rightarrow 3 \rightarrow 1$: cinco niveles tensoriales y cuatro transformaciones intermedias) hasta un único vector raíz. Es «híbrida» porque las contracciones tensoriales se realizan con TensorKrowch, mientras que las transformaciones entre niveles, el entrenamiento y la integración del modelo se gestionan con PyTorch.

TTNLevel Clase que representa un único nivel del árbol de la TTN. Se implementa como una red tensorial independiente que contrae sus bloques de tres nodos y devuelve un tensor de PyTorch ordinario. Rellena con nodos de *padding* el último grupo cuando el número de posiciones no es múltiplo de tres.

MPSClassifier Clase que envuelve una capa MPS (basada en la librería TensorKrowch) para usarla como clasificador. Recibe la secuencia de vectores locales y la contrae con una cadena MPS para producir las puntuaciones de clase. Corresponde a la opción de topología en cadena del proyecto.

MPSClassifier_1 Clase que envuelve la red HybridCustomTTN y le añade una cabeza de clasificación. Toma el vector raíz de dimensión `bond_dim` y lo transforma en las puntuaciones de las clases mediante una capa lineal o, opcionalmente, un pequeño perceptrón multicapa (MLP) con *dropout*.

PatchEmbedding Clase del primer *embedding* probado. Divide la imagen en cuadrados (*patches*) y proyecta cada uno de forma independiente con una capa lineal. Se descartó como versión final porque, al construir la secuencia, pierde la estructura bidimensional y la vecindad entre cuadrados contiguos.

TrigPixelEmbedding *Embedding* de tipo cuántico-inspirado que mapea cada valor de píxel (con `local_dim = 2`) al par $(\cos \theta, \sin \theta)$ con $\theta = \text{valor} \cdot \pi/2$. Implementa el mapa local de características $\phi(x)$ descrito en el marco teórico.

ParamNode Tipo de nodo de la librería TensorKrowch cuyos valores se registran como parámetros entrenables del modelo y reciben gradientes durante el entrenamiento. Los tensores de cada bloque de la TTN se declaran de este tipo; si se crearan como nodos ordinarios, no se optimizarían.

Nodos de *padding* (relleno) Nodos constantes y no entrenables, con vector fijo $[1, 0, \dots, 0]$, que completan un grupo de tres cuando faltan posiciones. Permiten que el bloque se contraiga sin aportar parámetros ni introducir información.

Stem convolucional Bloque inicial del AIMPatchEmbedding que sustituye el corte en cuadrados por una pila de bloques (convolución con paso 2, *BatchNorm* y *ReLU*).

Extrae características locales básicas y reduce a la mitad la resolución espacial en cada bloque.

Cabeza de clasificación Etapa final que convierte el vector raíz de la red tensorial en las puntuaciones por clase. En su versión de referencia es una única capa lineal ($\text{bond_dim} \rightarrow$ número de clases); opcionalmente, un MLP con una capa oculta, activación y *dropout*.

Embedding Etapa que transforma una imagen $[B, C, H, W]$ en la secuencia $[B, L, \text{local_dim}]$ que espera la red tensorial. Determina qué información espacial llega a la red y en qué forma.

Hiperparámetros y configuración

bond_dim Dimensión de enlace: dimensión común de los índices virtuales y del vector raíz de la TTN. Principal parámetro que controla la capacidad del modelo.

local_dim Dimensión local: tamaño del vector asociado a cada posición de entrada de la red tensorial, es decir, la dimensión de los índices físicos en el primer nivel.

patch_size En el `AIMPatchEmbedding` no representa el tamaño de un cuadrado, sino el factor total de reducción espacial del *stem*. Debe ser potencia de dos, y el número de bloques del *stem* es $\log_2(\text{patch_size})$.

aim_channels Número de canales que mantiene el mapa espacial dentro del *stem* y de los bloques AIM antes de proyectarse a *local_dim*.

expansion_ratio Factor por el que cada rama del bloque AIM2D expande el número de canales internamente, aumentando su capacidad de representación antes de comprimir de nuevo.

num_aim_blocks Número de bloques AIM2D que se aplican consecutivamente sobre el mapa producido por el *stem*.

Factor de escala residual Parámetro entrenable (inicializado a un valor pequeño, 0,1) que pondera la corrección que el bloque AIM2D suma a su entrada. Su inicialización pequeña hace que el bloque parta de un comportamiento próximo a la identidad.

Archivo de configuración (YAML) Fichero que define por completo un experimento (dataset, tamaño de imagen, tipo de *embedding*, *local_dim*, *bond_dim*, número de épocas, ritmo de aprendizaje, regularización, *scheduler* y semilla, entre otros), lo que permite reproducir o comparar experimentos sin depender de notas externas.

Entrenamiento, regularización y evaluación

Pipeline Estructura común del proyecto: `datos` $\rightarrow$ `transformaciones` $\rightarrow$ `DataLoader` $\rightarrow$ `modelo` $\rightarrow$ `pérdida` $\rightarrow$ `optimizador`. Separa responsabilidades entre la carga de datos, el modelo y el bucle de entrenamiento.

DataLoader Componente de PyTorch que agrupa las muestras del conjunto de datos en lotes (*batches*) para procesarlas de forma eficiente.

Batch (lote) Conjunto de imágenes que se procesan a la vez; primera dimensión (B) de los tensores del modelo.

Recorte (*crop*) Región de una imagen que contiene un único objeto, generada a partir de una anotación YOLO. El problema se reformuló como clasificación por instancia de estos recortes.

Data augmentation (aumento de datos) Conjunto de transformaciones aleatorias aplicadas solo en entrenamiento para que el modelo no memorice una única apariencia de cada objeto. Configurable mediante `augmentation_name` (`none`, `basic`, `light`); la configuración final `light` añade rotaciones de 7 grados y `RandomErasing` con probabilidad 0,10.

RandomErasing Transformación de aumento de datos que borra aleatoriamente una región de la imagen durante el entrenamiento.

Backpropagation (retropropagación) Algoritmo que calcula los gradientes de la función de coste respecto a los parámetros aplicando la regla de la cadena, propagando el error desde la salida hacia las capas inferiores.

LayerNorm Normalización por capas: reescala cada vector de canales para estabilizar su escala y distribución. Se emplea en el bloque `AIM2D` y en las transformaciones entre niveles de la TTN.

Softmax Función que normaliza las puntuaciones (*logits*) en una distribución de probabilidad. La calcula internamente `CrossEntropyLoss`, por lo que el modelo no la aplica en su capa de salida.

CrossEntropyLoss Función de pérdida de entropía cruzada de PyTorch, empleada para el entrenamiento. Combina internamente la normalización *softmax* y la comparación con la etiqueta correcta.

Label smoothing (suavizado de etiquetas) Técnica de regularización que relaja el objetivo, repartiendo una pequeña parte de la probabilidad entre las clases incorrectas para evitar un exceso de confianza. Se aplica solo en entrenamiento.

Weight decay Penalización que empuja suavemente los pesos hacia valores pequeños; la incorpora el optimizador AdamW. El proyecto añade además una penalización L2 explícita y configurable sobre las matrices de pesos.

Warmup (calentamiento) Mecanismo que, durante las primeras épocas, hace crecer gradualmente el ritmo de aprendizaje hasta su valor objetivo, evitando oscilaciones al inicio del entrenamiento.

ReduceLROnPlateau Planificador (*scheduler*) que reduce automáticamente el ritmo de aprendizaje cuando la pérdida de validación deja de mejorar durante varias épocas. El proyecto contempla también un descenso continuo tipo *cosine decay*.

Checkpoint (punto de control) Guardado del mejor modelo junto con el estado del optimizador y del planificador, que permite reanudar un entrenamiento interrumpido exactamente en las mismas condiciones.

Early stopping (parada temprana) Criterio que detiene el entrenamiento cuando la pérdida de validación deja de mejorar durante un número fijo de épocas. Se empleó en los experimentos de referencia con ResNet50.

Ajuste fino (*fine-tuning*) Esquema en el que el modelo parte de pesos preentrenados sobre ImageNet y todos ellos se actualizan durante el entrenamiento, adaptando las características visuales generales al problema concreto.

train_loss / eval_loss Pérdida (entropía cruzada) medida sobre el conjunto de entrenamiento y sobre el de validación, respectivamente. Ambas miden lo mismo y son directamente comparables, ya que **train_loss** no incluye el término de regularización.

Accuracy top-1 / top-5 Fracción de recortes en los que la clase real coincide con la más probable (top-1) o está entre las cinco más probables (top-5) para el modelo.

Precisión y exhaustividad (*precision / recall*) La precisión de una clase mide qué fracción de los recortes asignados a ella le pertenece realmente (penaliza falsos positivos); la exhaustividad mide qué fracción de los recortes que le pertenecen consigue recuperar (penaliza los que se escapan).

Media macro Promedio simple de una métrica (precisión, exhaustividad o F1) sobre las 25 clases, dando el mismo peso a cada una con independencia de su número de recortes.

Matriz de confusión Diagnóstico cuyas filas representan la clase real y sus columnas la clase predicha; la diagonal recoge los aciertos y los valores fuera de ella, las confusiones entre clases.

REFERENCIAS

- [1] Jairo Rojas Delgado and Rafael Trujillo-Rasúa. Firefly meta-heuristic algorithm applied to artificial neural network pre-training. 12, 02 2018.
- [2] Fred Van Veen and Stefan Leijnen. The neural network zoo. <https://www.asimovinstitute.org/neural-network-zoo>, 2019. Asimov Institute. Accessed: 2026-03-19.
- [3] Alejandro Mata Ali. Tensor Network Editor, May 2026.
- [4] Alejandro Mata Ali, Iñigo Perez Delgado, and Aitor Moreno Fdez. de Leceta. Quantum-inspired Techniques in Tensor Networks for Industrial Contexts. *arXiv e-prints*, page arXiv:2404.11277, April 2024.
- [5] Maolin Wang, Yu Pan, Zenglin Xu, Guangxi Li, Xiangli Yang, Danilo Mandic, and Andrzej Cichocki. Tensor networks meet neural networks: A survey and future perspectives. *arXiv preprint arXiv:2302.09019*, 2025. Version 3, revised 17 Mar 2025.
- [6] Andrzej Cichocki, Namgil Lee, Ivan Oseledets, Anh-Huy Phan, Qibin Zhao, and Danilo P Mandic. Tensor networks for dimensionality reduction and large-scale optimization part 1 low-rank tensor decompositions. *Foundations and Trends in Machine Learning*, 9(4-5):249–429, 2016.
- [7] E. Miles Stoudenmire and David J. Schwab. Supervised learning with quantum-inspired tensor networks, 2017.
- [8] Song Cheng, Lei Wang, and Pan Zhang. Supervised learning with projected entangled pair states. *Physical Review B*, 103(12):125117, 2021.
- [9] Norbert Schuch, Michael M. Wolf, Frank Verstraete, and J. Ignacio Cirac. Computational complexity of projected entangled pair states. *Physical Review Letters*, 98(14):140506, 2007.
- [10] G. Evenbly and G. Vidal. Tensor Network States and Geometry. *Journal of Statistical Physics*, 145(4):891–918, November 2011.
- [11] Chang Nie, Junfang Chen, and Yajie Chen. Deep tree tensor networks for image recognition, 2025. NeurIPS 2025 poster; arXiv:2502.09928.
- [12] José Ramón Pareja Monturiol, David Pérez-García, and Alejandro Pozas-Kerstjens. Tensorkrowch: Smooth integration of tensor networks in machine learning. *Quantum*, 8:1364, 2024.

- [13] Md Sakhawat Hossain et al. Real-world chemistry lab image dataset for equipment recognition across 25 apparatus categories. *Scientific Data*, 12(1747), 2025.
- [14] Jürgen Schmidhuber. Deep learning in neural networks: An overview. *Neural networks*, 61:85–117, 2015.
- [15] George Cybenko. Approximation by superpositions of a sigmoidal function. *Mathematics of Control, Signals and Systems*, 2(4):303–314, 1989.
- [16] Kurt Hornik. Approximation capabilities of multilayer feedforward networks. *Neural Networks*, 4(2):251–257, 1991.
- [17] Roger Penrose et al. Applications of negative dimensional tensors. *Combinatorial mathematics and its applications*, 1(221-244):3, 1971.
- [18] Jacob Biamonte and Ville Bergholm. Tensor networks in a nutshell, 2017.
- [19] Ulrich Schollwoeck. The density-matrix renormalization group in the age of matrix product states. *Annals of Physics - ANN PHYS N Y*, 326, 08 2010.
- [20] Tamara G Kolda and Brett W Bader. Tensor decompositions and applications. *SIAM review*, 51(3):455–500, 2009.
- [21] Ivan Oseledets. Tensor-train decomposition. *SIAM Journal on Scientific Computing*, 33:2295–2317, 09 2011.
- [22] G. Vidal. Entanglement renormalization. *Phys. Rev. Lett.*, 99:220405, Nov 2007.
- [23] F. Verstraete and J. I. Cirac. Renormalization algorithms for quantum-many body systems in two and higher dimensions. *arXiv preprint cond-mat/0407066*, 2004.

Capítulo 7. ANEXOS

En este capítulo se recoge el material complementario del trabajo. En particular, se enlaza el repositorio público que contiene el código fuente completo del proyecto, con el que se pueden reproducir los experimentos descritos en la metodología.

Todo el código desarrollado para este Trabajo de Fin de Grado se encuentra disponible en un repositorio público de GitHub. El repositorio incluye un archivo **README** con las instrucciones necesarias para instalar las dependencias y reproducir los experimentos presentados en este documento.

El repositorio puede consultarse en la siguiente dirección:

`https://github.com/Manuel-Arenas-Sanchez/Tensor-Network-Model-In-Depp-Learning`.
`git`

[PÁGINA INTENCIONADAMENTE EN BLANCO]